SECTION 5 | QUESTIONS WITH SOLUTIONS

The Mystery 8

How a neural network reads a picture, and how a handful of well-placed dots can fool it

HOW TO USE THIS HANDOUT

You do not need to know machine learning. Every new word is explained the first time it appears. Each topic starts with the idea in plain words, then a **Daily-Life Picture** that connects the idea to everyday life, then questions with complete, step-by-step solutions. The boxes marked **Additional Example** are extra, easy practice. Try each question yourself before you read its solution. There is no calculus and no coding: when a computer has to do hard maths, we simply tell you what it did. At the very end you will find one box, **What to remember**, that collects the key points of the whole handout.

THE MYSTERY IN ONE PICTURE

Look at the two pictures below. To your eyes, both are an 8. To the small neural network from Lesson 11, the left one is an 8 (it is 98% sure), but the right one is *not* an 8 (it is only 10% sure). The right picture is the left picture plus just 20 white dots. This week we play detective.



clean 8
$P(Y=8) = 0.98$
says 8

adversarial 8
$P(Y=8) = 0.10$
says NOT 8

Left: the clean 8. Right: the adversarial 8. The pink outlines mark the added white pixels. (Their exact positions here are only a sketch of the slide; you will find the real ones in the live lab.)

We will answer three questions: **Why** does the network miss? **How** do we find the pixels that fool it? And **what** does the network think an 8 really is?

OUR QUESTION FOR THE WHOLE SECTION

What is the fewest number of white pixels we must add to a picture of an 8, so that the network stops saying 8?

We will try three strategies: **try every pixel**, **read the weights**, and **use the slope**. Strategy 2 (read the weights) is the heart of the section.

WORDS WE USE (QUICK GLOSSARY)

Word	Easy meaning
Pixel	One tiny square dot of a picture. Our pictures are 28×28, so they have 784 pixels.
Brightness	The number that says how white a pixel is: 0 is black, 1 is white, and in between is grey.
Ink a pixel	Turn that pixel white.
Neuron	A tiny calculator. It looks at some numbers and gives back one number between 0 and 1.
Fires	A neuron *fires* (it says "yes") when its number is 0.5 or more.
Weight	A number that says how much one input matters to a neuron, and in which direction.
$P(Y=8)$	The network's chance that the picture is an 8. Near 1: "surely an 8". Near 0: "surely not".
Flip	$P(Y=8)$ drops below 0.5, so the network stops saying 8.
Clean, adversarial and fresh 8	The *clean 8* is the 8 we start with. The *adversarial 8* is the clean 8 plus 20 white pixels that fool the network. The *fresh 8* is a second 8 that we use to test our strategies.
h_1, h_2, h_3, h_4	The four hidden neurons of the small network. "Hidden" means in the middle, between the picture and the final answer.
Vetoes	The neurons h_3 and h_4. A veto is a neuron that can say "stop, this is not an 8".
Weight picture	A picture that shows one neuron's weight for every pixel. Ink on a **pink** pixel pushes the neuron up. Ink on a **blue** pixel pushes it down.

THE THREE BIG IDEAS OF THIS SECTION

THE RULE

The network says 8 when h_1 and h_2 are on and both vetoes (h_3 and h_4) are off.

THE WEAK SPOT

The vetoes are the easiest neurons to change: they are not far from switching on, and they have big weights into the output.

THE SLOPE

The backward pass that trains the weights also gives a slope for every pixel: a map of where ink hurts the network most.

These three ideas are explained slowly in the topics that follow, so do not worry if they sound strange now.

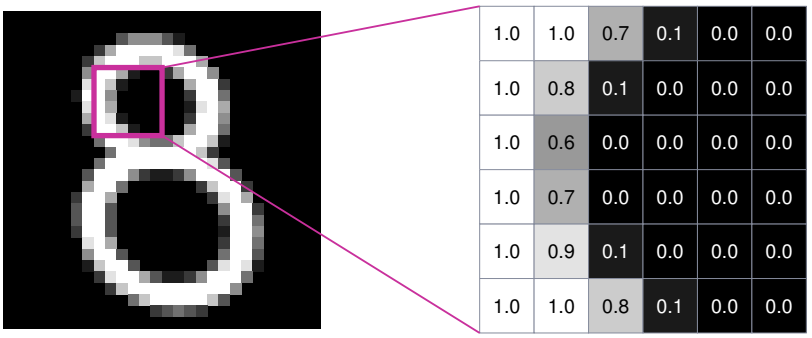
Topic 1 A picture is just a list of numbers

IN PLAIN WORDS

A computer cannot "see" a picture the way you do. It only understands numbers. So a picture is cut into a grid of tiny squares called **pixels**. Each pixel gets one number, its **brightness**: 0 means black, 1 means white, and a number in between means grey. Our digit pictures are 28 pixels wide and 28 pixels tall, so every picture is a list of $28 \times 28 = 784$ numbers. That list is the only thing the network ever receives.

DAILY-LIFE PICTURE

Picture a bathroom wall with 784 small tiles in a 28×28 square. Every tile is some shade of grey. If you write down the shade of each tile, row by row, a friend who has never seen the wall can rebuild the picture from your list. A cross-stitch pattern and an LED scoreboard work in the same way.



the whole picture
$28 \times 28 = 784$ pixels

zoom: $6 \times 6 = 36$ pixels
each number is a brightness

Zooming into the pink square. Every small square is one pixel, and the number inside it is its brightness (1.0 = white, 0.0 = black). The digits in this handout are redrawn by us in the style of the slides; they are not copies of the real data.

QUESTION 1 *counting*

How many numbers does the network receive for one picture? What brightness does a pure black pixel have, and what brightness does a pure white pixel have?

SOLUTION

Step 1: Count the pixels.

The picture has 28 rows and 28 columns. Each row has 28 pixels, so

$$28 \times 28 = 784 \text{ pixels.}$$

Step 2: One number per pixel.

Every pixel is described by exactly one brightness number, so the network receives 784 numbers.

Step 3: The two ends of the scale.

Black has brightness 0 and white has brightness 1. Grey pixels are in between, for example 0.5 is a medium grey.

ANSWER 784 numbers. Black is 0, white is 1.

QUESTION 2 *reading the zoom*

Look at the 6×6 zoom in the figure above. (a) Which column of the zoom is the brightest? (b) What do the zeros on the right side tell us? (c) Why can we not say that the network "sees a curve"?

SOLUTION

(a)

The left column is 1.0 in every row. These pixels are pure white: they are part of the thick white stroke of the 8.

(b)

The zeros on the right are black pixels. In the picture they are the empty space *inside* the top loop of the 8.

(c)

The pixels between white and black (for example 0.7, 0.8 and 0.1) are grey. They form the soft edge of the stroke. The network never receives the word "curve" or "loop". It receives only the list of numbers. Any shape it finds must come from calculations on those numbers.

ANSWER (a) The left column. (b) Black pixels, the empty inside of the loop. (c) The network gets only brightness numbers, not shapes.

ADDITIONAL EXAMPLE 1 *easy practice*

A tiny picture of a plus sign is 5×5 pixels. The white pixels (brightness 1) are the middle row and the middle column, and all other pixels are black (0). How many pixels are there in all, and how many of them are white?

0	0	1	0	0
0	0	1	0	0
1	1	1	1	1
0	0	1	0	0
0	0	1	0	0

ANSWER $5 \times 5 = 25$ pixels. The middle row has 5 white pixels and the middle column has 5, but the centre pixel is counted in both, so $5 + 5 - 1 = 9$ pixels are white and $25 - 9 = 16$ are black.

Topic 2 One neuron: a tiny voting calculator

IN PLAIN WORDS

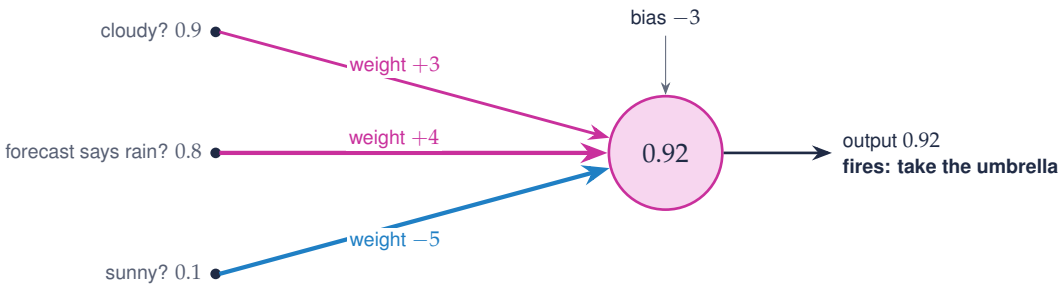
A **neuron** is a tiny calculator. It does three small jobs:

1. **Multiply** each input number by its **weight**. A weight says how much that input matters. A positive weight pushes the answer towards "yes". A negative weight pushes it towards "no". A big number (like 9 or -9) pushes hard, and a small one (like 0.1) hardly pushes at all.
2. **Add** all these pushes together, and add the neuron's own starting number, called the **bias**. The bias is the neuron's mood before it looks at any input.
3. **Squash** the total score into a number between 0 and 1 using an S-shaped curve (the **logistic curve**). Near 1 means "yes". Near 0 means "no".

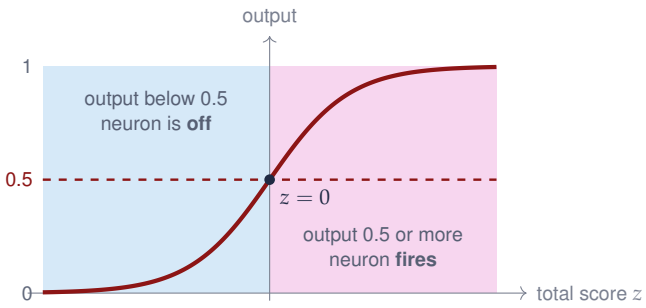
We say a neuron **fires** when its output is 0.5 or more. In the course you meet this recipe under the name *logistic regression*. So: **each neuron is a logistic regression on the layer before it.**

DAILY-LIFE PICTURE

You are deciding whether to carry an umbrella. You look at three things: *is it cloudy?*, *does the forecast say rain?*, *is it sunny?* Cloud and forecast push you towards "yes", so they get positive weights. Sunshine pushes you towards "no", so it gets a negative weight. The forecast matters most, so its weight is the biggest. Your bias is your natural laziness: you do not like carrying things, so before looking at anything your mood is already a little bit on the "no" side (a negative bias).



One neuron, the umbrella neuron. Pink arrows push towards "yes", the blue arrow pushes towards "no". Thicker arrow = bigger weight. (Made-up numbers, for illustration.)



The S-shaped (logistic) curve. It turns any score, however large or small, into a number between 0 and 1. A score of exactly 0 gives 0.5, which is the border between "off" and "fires".

score z	-6	-4	-2	0	2	4	6
output	0.002	0.02	0.12	0.50	0.88	0.98	0.998

Some values of the curve $\frac{1}{1+e^{-z}}$, so that you can read answers without a calculator.

QUESTION 3 *the umbrella neuron*

On a rainy morning the inputs are: cloudy $= 0.9$, forecast says rain $= 0.8$, sunny $= 0.1$. The weights are $+3$, $+4$, -5 and the bias is -3. What is the neuron's output? Does it fire?

SOLUTION

Step 1: Multiply each input by its weight.

input	value	weight	push
cloudy	0.9	$+3$	$0.9 \times 3 = +2.7$
forecast says rain	0.8	$+4$	$0.8 \times 4 = +3.2$
sunny	0.1	-5	$0.1 \times (-5) = -0.5$

Step 2: Add the pushes and the bias.

$$z = 2.7 + 3.2 - 0.5 + (-3) = 2.4$$

Step 3: Squash with the S-curve.

$$\text{output} = \frac{1}{1+e^{-2.4}} \approx 0.92$$

This matches the table above: a score of 2 gives 0.88 and a score of 4 gives 0.98, so 2.4 must be a little above 0.88.

Step 4: Fire or not?

$0.92 \geq 0.5$, so the neuron fires.

ANSWER The output is about 0.92. The neuron fires: take the umbrella.

QUESTION 4 *reading the S-curve*

A neuron has total score $z = 0$. What is its output? What if its score is -4? What if it is $+4$? In each case, does it fire?

SOLUTION

Use the table: $z = 0$ gives 0.50, $z = -4$ gives 0.02 and $z = 4$ gives 0.98.

ANSWER $z = 0$: output 0.50, it fires (the rule is "0.5 or more"), but only just. $z = -4$: output 0.02, it does not fire. $z = +4$: output 0.98, it fires strongly.

Note. A score of 0 is exactly on the border. That is why people say a neuron that is "not far from switching on" is easy to push over the line: a small push to its score is enough. We use this idea again in Topic 9.

ADDITIONAL EXAMPLE 2 *easy practice*

On a sunny afternoon: cloudy $= 0.1$, forecast says rain $= 0.0$, sunny $= 0.9$. Same weights ($+3$, $+4$, -5) and same bias (-3). Does the umbrella neuron fire?

ANSWER Pushes: $0.1 \times 3 = 0.3$, $0.0 \times 4 = 0$, $0.9 \times (-5) = -4.5$. Score: $0.3 + 0 - 4.5 - 3 = -7.2$. The output is about 0.0007, which is far below 0.5. The neuron does not fire: leave the umbrella at home.

Topic 3 Many neurons in layers: the full network

IN PLAIN WORDS

A **neural network** is many neurons arranged in **layers**. The first layer is simply the picture: 784 numbers. Every neuron in the next layer looks at *all* the numbers of the layer before, does its multiply–add–squash recipe, and passes its answer forward. The last layer gives the final answer.

The **full network** on the first slide has these layers:

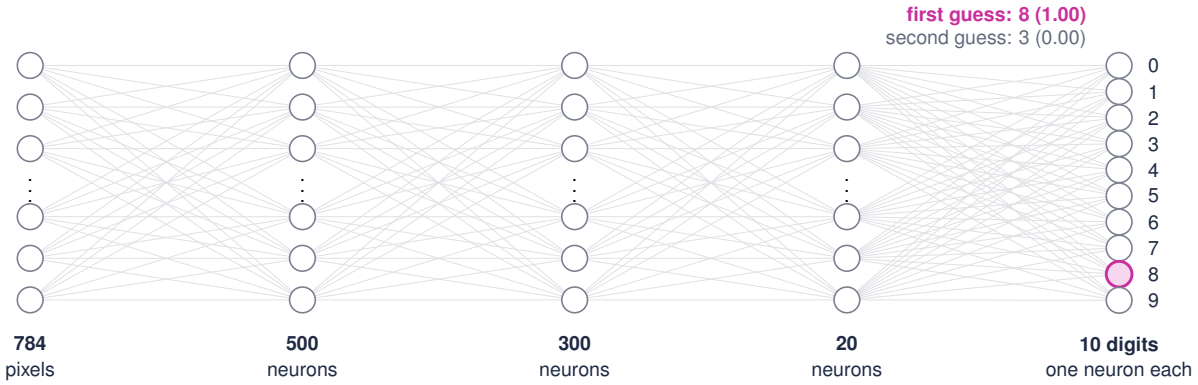
$$784 \text{ pixels} \rightarrow 500 \rightarrow 300 \rightarrow 20 \rightarrow 10 \text{ digits}.$$

The last layer has one neuron for each digit $0, 1, \ldots, 9$. The digit whose neuron gives the biggest number is the network's **first guess**. On the slide the first guess is **8** with 1.00, and the second guess is 3 with 0.00. So the network is almost completely sure.

Every neuron has its own weights and its own bias. All these numbers together are called the network's **weights and biases**. The full network has 549,030 of them.

DAILY-LIFE PICTURE

Think of a big office. 784 junior workers each look at one tiny tile of the picture and report a number. 500 team leaders each read all 784 reports and write one short summary. 300 managers read the 500 summaries. 20 directors read the 300 manager notes. Finally 10 decision makers, one for each digit, read the 20 director notes, and each says how sure it is that the picture is *their* digit. The loudest voice wins.



The full network. Only a few neurons of each layer are drawn; the dots (⋮) stand for all the others. Every neuron is joined to every neuron of the next layer. The pink neuron is the digit 8.

QUESTION 5 *counting the numbers*

The slide says the full network has 549,030 weights and biases. Check this number. (Rule: a neuron has **one weight for every input it receives**, plus **one bias**.)

SOLUTION

Step 1: Count layer by layer.

Take the layer $784 \to 500$. Each of the 500 neurons receives 784 inputs, so it has 784 weights and 1 bias.

$$\text{weights: } 784 \times 500 = 392{,}000 \qquad \text{biases: } 500 \qquad \text{total: } 392{,}500$$

Do the same for each layer.

layer	weights	biases	total
$784 \to 500$	392,000	500	392,500
$500 \to 300$	150,000	300	150,300
$300 \to 20$	6,000	20	6,020
$20 \to 10$	200	10	210
all layers			**549,030**

Step 2: Add the totals.

$$392{,}500 + 150{,}300 + 6{,}020 + 210 = 549{,}030$$

ANSWER The count is correct: 549,030 weights and biases. Most of them sit in the very first layer, because every neuron there must look at all 784 pixels.

QUESTION 6 *reading the output layer*

The slide says: "First guess: 8 (1.00). Second guess: 3 (0.00)." What does this tell us about the network's opinion of the picture?

SOLUTION

The last layer has 10 neurons, one per digit. The neuron for 8 outputs 1.00, which means "I am almost completely sure it is an 8". The next best neuron is the one for 3, and it outputs only 0.00 (it is so small that it rounds to zero). So there is no real competition: the network has no doubt.

ANSWER The network is almost 100% sure that the picture is an 8, and it gives essentially no chance to any other digit.

ADDITIONAL EXAMPLE 3 *easy practice*

A tiny network has 4 inputs, then a layer of 3 neurons, then a layer of 2 neurons ($4 \to 3 \to 2$). How many weights and biases does it have?

ANSWER Layer $4 \to 3$: $4 \times 3 = 12$ weights and 3 biases, so 15. Layer $3 \to 2$: $3 \times 2 = 6$ weights and 2 biases, so 8. Total: $15 + 8 = 23$.

Topic 4 Where do the weights come from? Training and MLE

IN PLAIN WORDS

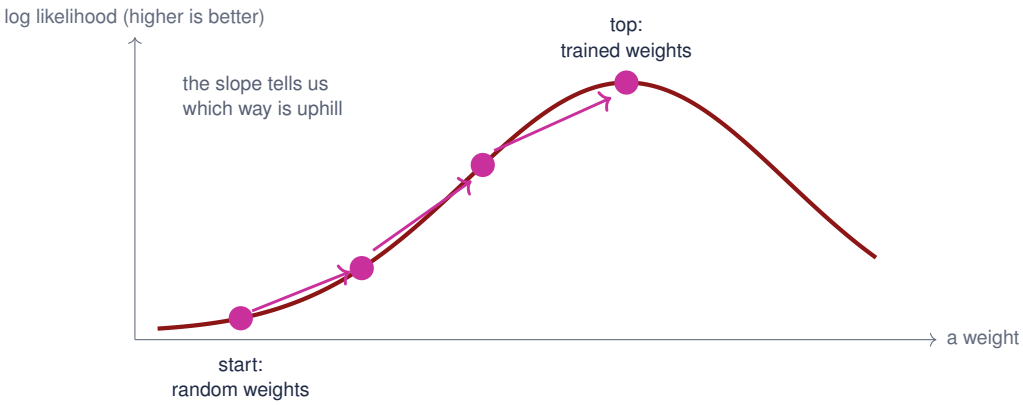
Nobody types the 549,030 numbers by hand. They are **learned**. We show the network thousands of **training digits**, each with the correct answer. At the start the weights are random, so the network guesses badly. Then training changes the weights, a little at a time, so that the network gives a *higher chance to the correct answers*.
To measure "how good are the weights?" we use a score:

- The **likelihood** is the chance the network gave to the correct answers of *all* the training digits together. For several digits, we multiply their chances.
- Multiplying many small numbers gives a tiny, awkward number. So we take the **logarithm** (log). It turns a product into a sum and keeps the order: a bigger likelihood has a bigger log. The **log likelihood** is always 0 or negative, and *closer to* 0 *is better*.

Training **climbs** the log likelihood. This idea is called **maximum likelihood estimation** (MLE): *choose the weights that make the training digits as likely as possible.* The computer finds the uphill direction for every weight with the **backward pass**. The uphill direction of a number is called its **slope**.

DAILY-LIFE PICTURE

You are on a foggy hill and want to reach the top. You cannot see it, but you can feel the ground under your feet. If the ground tilts up in front of you, you take a small step that way. Then you feel the ground again and repeat. Training does exactly this, but the "ground" is the log likelihood, and instead of left–right and forward–back there are 549,030 directions, one for each weight.



Training as hill climbing. A hill with one weight is easy to draw. The real hill has 549,030 directions, but the idea is the same: feel the slope, step uphill, repeat.

QUESTION 7 *likelihood and log likelihood*

A tiny training set has 3 digits. Before training, the network gives the correct answer these chances: 0.9, 0.8 and 0.5. After training, it gives 0.95, 0.9 and 0.8. Work out the likelihood and the log likelihood before and after. Which weights are better?

SOLUTION

Step 1: Likelihood = multiply the chances.

$$\text{before: } 0.9 \times 0.8 \times 0.5 = 0.36 \qquad \text{after: } 0.95 \times 0.9 \times 0.8 = 0.684$$

Step 2: Log likelihood = add the logs.

We use the natural logarithm ln (calculator button "ln").

	digit 1	digit 2	digit 3	log likelihood
before	$\ln 0.9 = -0.105$	$\ln 0.8 = -0.223$	$\ln 0.5 = -0.693$	-1.022
after	$\ln 0.95 = -0.051$	$\ln 0.9 = -0.105$	$\ln 0.8 = -0.223$	-0.380

Step 3: Compare.

The likelihood went up ($0.36 \to 0.684$) and the log likelihood moved closer to zero ($-1.022 \to -0.380$). Both say the same thing: the trained weights are better.

ANSWER Before: likelihood 0.36, log likelihood ≈ -1.02. After: likelihood 0.684, log likelihood ≈ -0.38. The weights after training are better.

QUESTION 8 *explain to a friend*

The slide asks: "How were the weights chosen?" Answer it in two or three sentences for a friend who knows no machine learning.

SOLUTION

A good answer has three ideas: (1) the weights are *learned*, not typed in; (2) we show many training digits with their correct labels; (3) we keep changing the weights, using the slope, so the chance that the network gives to the right answers goes up.

ANSWER "The network started with random weights. We showed it thousands of digits with the correct answers and kept nudging every weight in the direction that makes the correct answers more likely. This is called maximum likelihood: pick the weights that make the training digits as likely as possible."

ADDITIONAL EXAMPLE 4 *easy practice*

A training set has only 2 digits. Before training the correct-answer chances are 0.7 and 0.6. After training they are 0.9 and 0.8. Find the likelihood before and after, and say which is better.

ANSWER Before: $0.7 \times 0.6 = 0.42$. After: $0.9 \times 0.8 = 0.72$. The logs are $\ln 0.42 \approx -0.87$ and $\ln 0.72 \approx -0.33$. The second is closer to 0 (and larger), so the weights after training are better.

Topic 5 The small network we will attack

IN PLAIN WORDS

The full network is too big to study by hand (549,030 numbers). So for the rest of this section we use a **small network**:

$$784 \text{ pixels} \;\rightarrow\; 4 \text{ hidden neurons} \;\rightarrow\; 1 \text{ output neuron.}$$

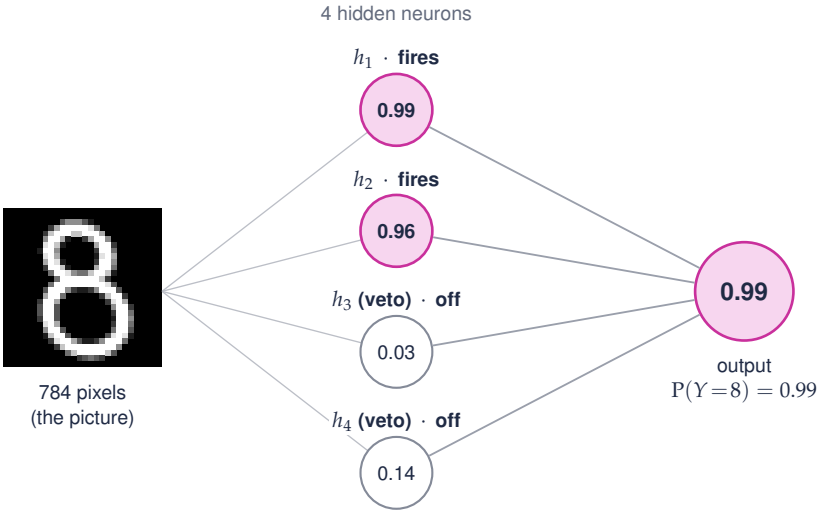
The four middle neurons are called **hidden** because they sit between the picture and the final answer. We call them h_1, h_2, h_3, h_4. The single output neuron answers one question only: *is this picture an 8?* Its answer is written $\mathrm{P}(Y = 8)$, which you read as "the chance that the digit Y is an 8".

On the slide, the 8 gives $\mathrm{P}(Y = 8) = 0.99$, and the four hidden neurons give 0.99, 0.96, 0.03, 0.14. Remember: a neuron **fires** when its number is 0.5 or more.

The rule from Lesson 11. The network says "8" when h_1 and h_2 are **on** and both **vetoes** (h_3 and h_4) are **off**. A *veto* is a neuron that says "stop, this is not an 8".

DAILY-LIFE PICTURE

A school trip needs permission. Two teachers (h_1 and h_2) must both sign. But two guards at the gate (h_3 and h_4) each have the power to stop the trip. The trip goes ahead only if *both teachers sign and neither guard objects*. The guards are the vetoes: it is not enough to collect the signatures, you must also keep the guards quiet.



The small network. The two pink neurons fire (≥ 0.5), the two white ones do not. The rule is satisfied, so the network says "8" with $P(Y=8) = 0.99$.

QUESTION 9 *using the rule*

For the 8 on the slide the hidden neurons give 0.99, 0.96, 0.03 and 0.14. Which neurons fire? Does this picture follow the rule for "8"?

SOLUTION

Step 1: Compare each neuron with 0.5.

neuron	value	≥ 0.5?	meaning
h_1	0.99	yes	fires
h_2	0.96	yes	fires
h_3 (veto)	0.03	no	off
h_4 (veto)	0.14	no	off

Step 2: Check the rule.

The rule asks for h_1 on, h_2 on, h_3 off and h_4 off. All four conditions are true.

ANSWER h_1 and h_2 fire, and the two vetoes stay off. The rule is satisfied, so the network says 8 (with $P(Y=8) = 0.99$).

QUESTION 10 *what about a 3?*

The slide also asks: "Which neurons fire for an 8? For a 3?" What can we say about a 3?

SOLUTION

The slide shows the numbers only for the 8; for the 3 it leaves the question to the room. What we can say is this:

- A 3 is *not* an 8, so the network should answer "not 8".
- By the rule, that happens when **at least one** condition fails: h_1 is off, or h_2 is off, or a veto (h_3 or h_4) is on.
- *Our reading, not stated on the slides:* which one fails depends on the picture. Click the digit 3 in the live lab and watch the four neurons to find out.

ANSWER For a 3, at least one condition of the rule must fail, so the network says "not 8". The live lab shows which neuron changes.

QUESTION 11 *small versus full*

Check that the small network has 3,145 weights and biases. How many times smaller is it than the full network?

SOLUTION

Step 1: Layer $784 \to 4$.

Each of the 4 neurons has 784 weights and 1 bias: $784 \times 4 = 3{,}136$ weights and 4 biases, so 3,140.

Step 2: Layer $4 \to 1$.

The output neuron has 4 weights and 1 bias, so 5.

Step 3: Add and compare.

$$3{,}140 + 5 = 3{,}145 \qquad \frac{549{,}030}{3{,}145} \approx 174.6$$



ANSWER 3,145 is correct. The small network is about 175 times smaller than the full one, which makes it easy to study neuron by neuron.

ADDITIONAL EXAMPLE 5 *easy practice*

Another picture gives hidden values $h_1 = 0.97$, $h_2 = 0.20$, $h_3 = 0.05$, $h_4 = 0.10$. Does it follow the rule for "8"?

ANSWER $h_1 = 0.97$ fires, but $h_2 = 0.20$ is below 0.5, so h_2 is off. The rule needs h_2 on, so this picture does *not* follow it: the network says "not 8". (The rule is a handy summary of the output neuron's calculation; Topic 9 shows that calculation.)

Topic 6 The game "Still an 8?"

IN PLAIN WORDS

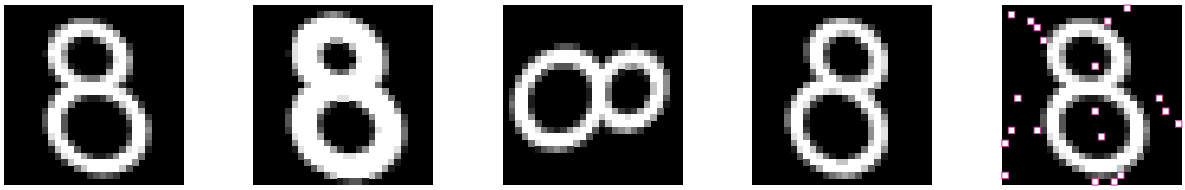
Let us find out how tough the network is. We start from a clean 8 where the network is 98% sure ($\mathrm{P}(Y=8) = 0.98$). Then we change the picture in four ways. After each change we ask: **will it still say 8?** The slide reminds us of the rule: *the network says 8 when* $\mathrm{P}(Y=8) \geq 0.5$. The four changes are:

1. draw it with a **thicker pen**;
2. turn it **sideways**;
3. **tilt** it by 15°;
4. **sprinkle** 20 **random white pixels** on it.

To a person, all four pictures are still clearly an 8.

DAILY-LIFE PICTURE

Your friend's handwriting is messy, or he writes with a thick marker, or the page is slightly crooked, or there is some dust on the scanner glass. You still read the word without any trouble. The game asks: is the network as relaxed as you are?



clean 8 $\mathrm{P}(Y=8) = 0.98$ | **thicker pen** $\mathrm{P}(Y=8) = ?$ | **sideways** $\mathrm{P}(Y=8) = ?$ | **tilt** 15° $\mathrm{P}(Y=8) = ?$ | **20 random pixels** $\mathrm{P}(Y=8) = ?$

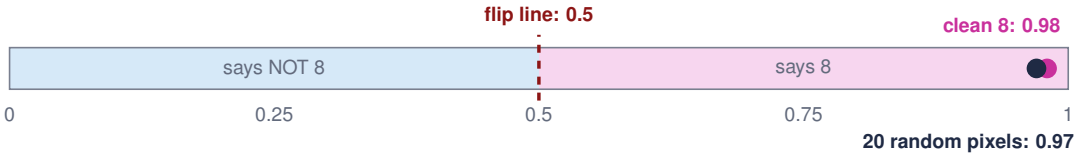
The clean 8 and the four changes of the game. Which of them will the network still read as an 8?

PLACE YOUR BET

Before you look at any answer, write your bet for each row: "still 8" or "not 8". Then check the real result in the live lab.

change	your bet	result
thicker pen	_____	shown in the live lab
sideways	_____	shown in the live lab
tilt 15°	_____	shown in the live lab
20 random white pixels	_____	$P(Y=8) = 0.97$, says 8 (next slide)

Note. The slides print the answer only for the last row. We do not invent numbers for the other three: the live lab shows them.



The "flip line". Any picture with $P(Y=8)$ at 0.5 or more is read as an 8. Below 0.5 the network says "not 8". After 20 random pixels the picture has hardly moved.

QUESTION 12 *using the rule*

Using the rule "says 8 when $P(Y=8) \geq 0.5$", what does the network say for $P(Y=8) = 0.97$, $P(Y=8) = 0.62$, $P(Y=8) = 0.50$ and $P(Y=8) = 0.49$?

SOLUTION

Compare each value with the flip line 0.5.

$P(Y=8)$	network says	
0.97	8	(far above 0.5)
0.62	8	(above 0.5, but not very sure)
0.50	8	(the rule says "0.5 or more")
0.49	not 8	(just below the line)

ANSWER $0.97 \to 8$, $0.62 \to 8$, $0.50 \to 8$, $0.49 \to$ not 8.

Note. Notice how thin the line is. 0.50 and 0.49 are almost the same number, but they give different answers. We will use this later: a picture that sits just above the line can be flipped by a very small change.

ADDITIONAL EXAMPLE 6 *easy practice*

After some changes, four pictures have $P(Y=8) = 0.30$, 0.85, 0.50 and 0.05. Which ones does the network read as 8?

ANSWER 0.85 and 0.50 are 0.5 or more, so these two are read as 8. 0.30 and 0.05 are below 0.5, so these are "not 8".

Topic 7 The same 8, 20 different pixels

IN PLAIN WORDS

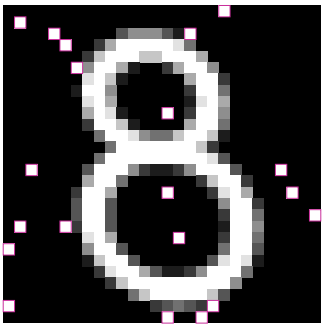
Now the surprise. We add exactly 20 white pixels to the same 8 in two different ways:

- **20 random pixels:** $P(Y=8) = 0.97$. The network says 8.
- **20 chosen pixels:** $P(Y=8) = 0.10$. The network says *not* 8.

Same 8, same number of added pixels. Only the *place* of the pixels is different. To your eyes both pictures are still an 8. So the slide asks: **"You'd still read an 8. Why doesn't it?"**
A first idea (the full answer comes in Topics 10 to 17): you recognise the *shape*. The network does not. It only adds up weighted votes from the pixels. Some pixels have a big weight, and a few well-placed pixels can change the total a lot.

DAILY-LIFE PICTURE

A page has price tags. If twenty random ink drops fall on it, you can still read every price. But one small ink stroke, placed on exactly the right spot, can turn "Rs 10" into "Rs 70". The number of marks is not what matters. The *place* of the marks is.



20 random pixels
$P(Y=8) = 0.97$
says 8



20 chosen pixels
$P(Y=8) = 0.10$
says NOT 8

Left: random pixels (the pink outlines mark them). Right: chosen pixels. The positions here are redrawn to look like the slide, not copied from the live lab.

QUESTION 13 *place or number?*

Compare the two pictures. By how much did $P(Y=8)$ drop in each case, starting from the clean 8 ($P(Y=8) = 0.98$)? How many times bigger is the second drop?

SOLUTION

Step 1: The drop for random pixels.

$$0.98 - 0.97 = 0.01$$

Step 2: The drop for chosen pixels.

$$0.98 - 0.10 = 0.88$$

Step 3: Compare.

$$\frac{0.88}{0.01} = 88$$

Both pictures got exactly 20 extra pixels, so the number of pixels cannot be the reason for the difference.

ANSWER Random pixels lower $P(Y=8)$ by 0.01. Chosen pixels lower it by 0.88, which is 88 times more. What matters is *where* the pixels go.

Note. This is the heart of the section. If place matters so much, we need a good **strategy** for choosing the places. That is what Topics 10 to 17 are about.

ADDITIONAL EXAMPLE 7 *easy practice*

Three pictures have $P(Y=8) = 0.98$ (the clean 8), $P(Y=8) = 0.97$ (20 random pixels) and $P(Y=8) = 0.10$ (20 chosen pixels). Put them from *most sure it is an 8* to *least sure*, and say which ones the network reads as 8.

ANSWER Order: 0.98, then 0.97, then 0.10. The first two are above 0.5, so the network reads them as 8. The last one is below 0.5, so it says "not 8".

Topic 8 Adversarial examples and the smallest change

IN PLAIN WORDS

An **adversarial example** is an input that has been nudged *on purpose*, by a small amount, so that the network gives a different answer. People looking at it see almost no change.
The slide gives two real stories, both from 2018: a **stop sign** that a network read as "Speed Limit 45", and a toy **turtle** that a network read as a "rifle". In our game, the adversarial 8 is the clean 8 plus 20 chosen white pixels.
Now the real question of this section: **what is the smallest change that works?** Turn as few pixels white as you can, until $P(Y=8) < 0.5$ and the network says "not 8".

DAILY-LIFE PICTURE

Imagine a few small stickers on a stop sign. Any driver still sees a stop sign at once. But the network inside a camera may read the same sign as a speed limit. The change is tiny for people and large for the network.



The idea of an adversarial example (a simple drawing of the first story on the slide, 2018). The second story is the toy turtle that was read as a rifle.



Twenty well-chosen pixels move the picture from far above the flip line to far below it.

QUESTION 14 *why the fewest?*

The slide says: "Turn as few pixels white as you can. Make it stop saying 8." In terms of $P(Y=8)$, what exactly must happen? Why is it interesting to use *as few* pixels as possible?

SOLUTION

Step 1: What must happen.

We start at $P(Y=8) = 0.98$. We must push it below the flip line: $P(Y=8) < 0.5$. That is a drop of more than $0.98 - 0.5 = 0.48$.

Step 2: Why few pixels.

Our reading, not stated on the slides: the fewer pixels we change, the smaller and less noticeable the change is, so a small number shows how fragile the network is. A big number would show nothing: if you scribble all over a picture, any reader will be confused.

ANSWER $P(Y=8)$ must fall from 0.98 to below 0.5 (a drop of more than 0.48). Using the fewest pixels shows how easy it is to fool the network.

QUESTION 15 *guess the minimum*

The slide asks: "Take a guess: what is the minimum number of pixels to flip the digit?" We already know that 20 chosen pixels are enough. What else can we say about the minimum before we try anything?

SOLUTION

Step 1: An upper limit.

The adversarial 8 has 20 chosen pixels and $P(Y=8) = 0.10$, so 20 pixels do flip it. The minimum is **at most 20**.

Step 2: A lower limit.

With 0 added pixels we have the clean 8 with $P(Y=8) = 0.98$, and the network says 8. So 0 pixels do not flip it. The minimum is **at least 1**.

Step 3: What is left.

The true answer is somewhere from 1 to 20. Is it 19? Is it 5? We find it with the strategies of Topics 10 to 13.

ANSWER The minimum is a whole number from 1 to 20. The exact value comes from the live lab "Watch it flip" and from the strategies of Topics 10 to 13.

PLACE YOUR BET

Write your guess for the minimum number of white pixels that flips the clean 8: _____

In class, each student posts this bet in the chat *before* the live lab "Watch it flip". In that lab you click outside the picture (or press →) to add the next worst pixel, the one that lowers $P(Y=8)$ the most. You can click inside the picture to add your own pixel, and press ← to undo. When the digit flips, press "Move on".

ADDITIONAL EXAMPLE 8 *easy practice*

(Made-up numbers, for illustration.) A picture starts at $P(Y=8) = 0.98$. After 3 added pixels it is 0.90, after 6 pixels 0.55, after 7 pixels 0.47. After how many pixels did it flip?

ANSWER It flips when $P(Y=8)$ drops below 0.5. At 6 pixels it is still 0.55, which says 8. At 7 pixels it is $0.47 < 0.5$, so it flips at **7** pixels.

Topic 9 Our three goals, and the job we are given

IN PLAIN WORDS

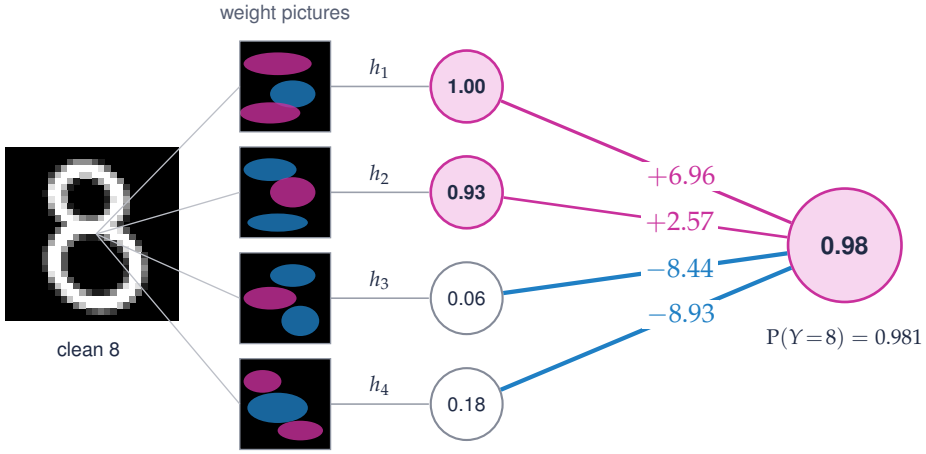
The slides set three **learning goals** for this section:

1. **Why did it miss?** Why does the network fail on the adversarial 8?
2. **How do you find the pixels that fool it?**
3. **What does it think an 8 is?**

Then comes our job: **come up with a strategy for how to place the pixels.** In the live lab you see the small network at work. You click on pixels of the 8 to turn them white (a counter shows how many pixels you have inked, and "Reset" starts again). On the right you see the four hidden neurons, each with a small **weight picture** next to it, and the output neuron.

DAILY-LIFE PICTURE

You are the coach of a school quiz team and you want to understand the judges. You cannot read their minds, but you can see how loudly each judge shouts "yes" or "no", and how much weight the head judge gives to each of them. Your job is to find which small change in the answer sheet makes the judges shout the loudest "no".



The wiring of the small network, as on the slide. The numbers on the lines are the output neuron's weights: pink = pushes towards 8, blue = pushes against 8. The weight pictures here are only schematic drawings; the real ones are in the live lab.

QUESTION 16 *how is 0.981 made?*

The hidden neurons are $h_1 = 1.00$, $h_2 = 0.93$, $h_3 = 0.06$, $h_4 = 0.18$. The output weights are $+6.96$, $+2.57$, -8.44, -8.93. Show how the output neuron arrives at $\mathrm{P}(Y=8) = 0.981$.

SOLUTION

Step 1: Every hidden neuron casts a vote (value $\times$ weight).

neuron	calculation	vote
h_1	$1.00 \times (+6.96)$	$+6.96$
h_2	$0.93 \times (+2.57)$	$+2.39$
h_3	$0.06 \times (-8.44)$	-0.51
h_4	$0.18 \times (-8.93)$	-1.61

Step 2: Add the votes.

$$6.96 + 2.39 - 0.51 - 1.61 \approx +7.24$$

(The exact total is 7.236. Adding the rounded numbers in the table gives 7.23; that is only rounding.)

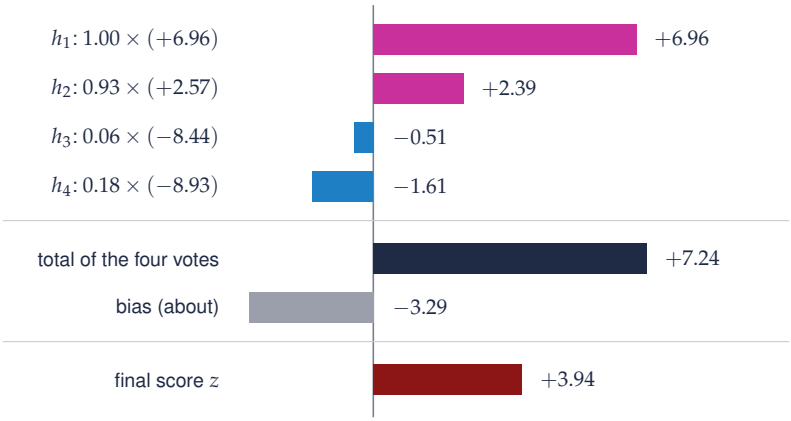
Step 3: Add the output neuron's own bias.

Like every neuron, the output neuron has a bias, but the slide does not show it. *Our calculation, not on the slides:* for the final answer to be 0.981 the bias must be about -3.29. This gives the final score

$$z \approx 7.24 - 3.29 = 3.94.$$

Step 4: Squash with the S-curve.

$$\mathrm{P}(Y=8) = \frac{1}{1+e^{-3.94}} \approx 0.981$$



ANSWER The four votes add up to about $+7.24$. The output neuron's bias takes away about 3.29, leaving a score of about $+3.94$. The S-curve turns this into $\mathrm{P}(Y=8) \approx 0.981$.

QUESTION 17 *what if a veto fires?*

Suppose h_4 jumps from 0.18 to 1.00 (it fires), and nothing else changes. What happens to the score and to $\mathrm{P}(Y=8)$?

SOLUTION

Step 1: How much does the vote of h_4 change?

Before: $0.18 \times (-8.93) = -1.61$. After: $1.00 \times (-8.93) = -8.93$. The change is

$$0.82 \times (-8.93) \approx -7.32.$$

Step 2: New score.

$$z \approx 3.94 - 7.32 = -3.38$$

Step 3: Squash.

$$\mathrm{P}(Y=8) = \frac{1}{1+e^{3.38}} \approx 0.03$$

ANSWER The score drops from $+3.94$ to about -3.38, and $\mathrm{P}(Y=8)$ falls from 0.98 to about 0.03. One veto that fires is enough to make the network say "not 8".

Note. This is the idea behind the "weak spot" of the network. A veto has a very big weight (-8.44 or -8.93), so when it switches on it can overrule the two supporters. In Topics 11 to 13 we learn how to find the pixels that switch a veto on.

ADDITIONAL EXAMPLE 9 *easy practice*

Suppose instead that h_3 jumps from 0.06 to 1.00, and nothing else changes. What is $P(Y=8)$ now?

ANSWER The vote of h_3 changes by $0.94 \times (-8.44) \approx -7.93$. The new score is $3.94 - 7.93 \approx -3.99$, so $P(Y=8) = \frac{1}{1+e^{3.99}} \approx 0.02$. Again the network says "not 8".

Topic 10 Strategy 1: try everything

IN PLAIN WORDS

The most direct idea is to simply **try everything**:

1. Take the picture. Pick one pixel and imagine turning it white. Ask the network: "what is $P(Y=8)$ now?"
2. Do this for *every* pixel, one at a time.
3. Keep the pixel that lowers $P(Y=8)$ the most. We call it the **worst pixel** (worst for the network).
4. Repeat with the next pixel, and the next, until $P(Y=8) < 0.5$. Then the digit has **flipped**.

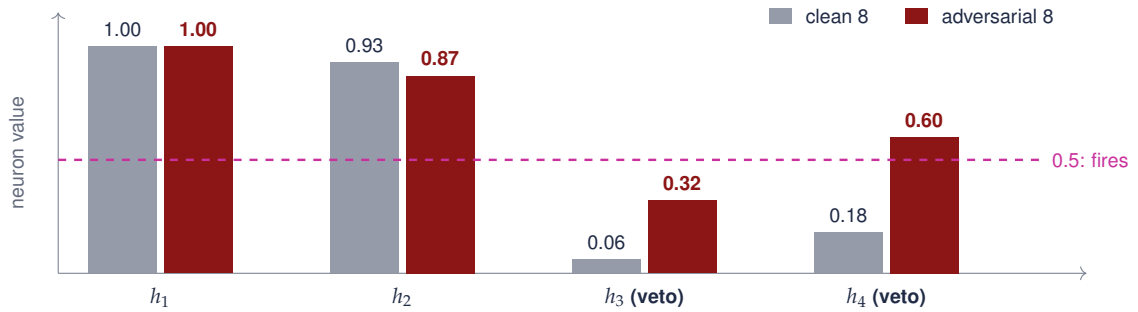
This is what the live lab **"Watch it flip"** does: each time you click outside the picture (or press $\rightarrow$) it adds the next worst pixel. The course notes give **14 pixels** as the fewest that can flip the clean 8 when we may only use white pixels (our reading: this is the number you reach in the lab). The adversarial 8 on the slides uses 20 pixels so that we can compare it fairly with the 20 random pixels of Topic 7.

DAILY-LIFE PICTURE

You have a big key ring with 784 keys and only one of them opens the lock. The "try everything" method takes the first key, tries it, takes the second key, tries it, and so on. It will always work, but you may need hundreds of tries. And in our game you must do this again for every pixel you add!

TRY IT IN THE LIVE LAB

In the lab **"Strategy #1: Try Everything"** the computer first tries every pixel for you, and then each press of $\rightarrow$ adds one more pixel. Watch the four hidden neurons while you do this: the ones that change are the **vetoes**. h_4 starts to fire and h_3 rises. Before the lab, you can also try some pixels by hand. You will see that **one pixel barely moves** $P(Y=8)$.



The hidden neurons before (clean 8) and after (adversarial 8) the 20 pixels are added. The two supporters hardly change. The two vetoes both move up, and h_4 crosses the 0.5 line.

QUESTION 18 *how many tries?*

Strategy 1 must test every pixel to find the worst one, and it must do this again for every pixel it adds. About how many tries does it need to flip the clean 8 with 14 pixels? Roughly how many multiplications is that for the small network, and for the full network?

SOLUTION

Step 1: Tries for one pixel.

There are 784 pixels, but the clean 8 already covers about 137 of them, and a pixel that is already white cannot be inked again. So the first search tests $784 - 137 = 647$ black pixels. For the second pixel 646 remain, then 645, and so on. (The exact count depends on the picture.)

Step 2: Add them up for 14 pixels.

$$647 + 646 + \cdots + 634 = 14 \times \frac{647 + 634}{2} = 14 \times 640.5 = 8{,}967 \text{ tries.}$$

Step 3: What does one try cost?

One try means running the network once. A run uses each weight about once, so it costs about as many multiplications as the network has weights: 3,145 for the small network and 549,030 for the full one.

network	multiplications per try	for 8,967 tries
small (3,145)	$\approx$ 3,145	$\approx$ 28 million
full (549,030)	$\approx$ 549,030	$\approx$ 4.9 billion

ANSWER About 9,000 tries (8,967). For the small network that is about 28 million multiplications, which a computer does in a blink. For the full network it would be about 4.9 billion. Try everything works, but it becomes expensive very fast.

Note. This is why people look for cheaper strategies. A later slide ("Try Everything vs Calculus") says that one search for the most confusing pixel takes 655 tries. We meet it again in Topic 17.

QUESTION 19 *reading the inside of the network*

On the slide "Strategy #1: Try Everything" the adversarial 8 gives $P(Y=8) = 0.10$. The hidden neurons are $h_1 = 1.00$, $h_2 = 0.87$, $h_3 = 0.32$, $h_4 = 0.60$. For the clean 8 they were 1.00, 0.93, 0.06, 0.18. Which neurons changed, and by how much? Did any neuron cross the 0.5 line?

SOLUTION

Step 1: Change = after − before.

neuron	before	after	change	fires after?
h_1	1.00	1.00	0.00	yes (as before)
h_2	0.93	0.87	-0.06	yes (as before)
h_3 (veto)	0.06	0.32	$+0.26$	no
h_4 (veto)	0.18	0.60	$+0.42$	**yes**

Step 2: Read the table.

The supporters h_1 and h_2 hardly moved. The vetoes moved the most. h_3 rose a lot but stays below 0.5. h_4 rose from 0.18 to 0.60 and crossed the line, so a veto now fires.

ANSWER The vetoes changed most: h_4 by $+0.42$ (it now fires) and h_3 by $+0.26$ (still off). h_2 dropped by only 0.06 and h_1 did not change. A veto fires, so the rule for "8" fails and the network says "not 8".

QUESTION 20 *why does one pixel do so little?*

(Made-up numbers, for illustration.) Imagine that each helpful pixel we ink adds $+0.5$ to the *score* of h_4. Right now $h_4 = 0.18$. Use the S-curve table of Topic 2 to find h_4's score. Then find out how many pixels are needed before h_4 fires.

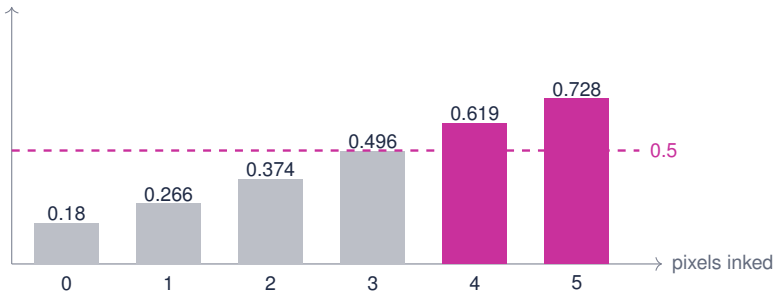
SOLUTION

Step 1: Find the score of h_4.

In the table of Topic 2, score -2 gives 0.12 and score 0 gives 0.50. So 0.18 must be between them. A calculator gives $\ln(0.18/0.82) = -1.52$ (this is the S-curve run backwards).

Step 2: Add 0.5 for every pixel.

pixels inked	0	1	2	3	4	5
score of h_4	-1.52	-1.02	-0.52	-0.02	$+0.48$	$+0.98$
value of h_4	0.18	0.27	0.37	0.496	0.62	0.73



Made-up example: h_4 rises by a small step with every pixel and crosses the line only at the fourth pixel.

Step 3: When does it fire?

After 3 pixels it is 0.496, just *below* 0.5. After 4 pixels it is 0.62, so it fires.

ANSWER The score of h_4 starts at about -1.52. The first pixel moves h_4 only from 0.18 to 0.27. It takes 4 pixels before h_4 fires.

Note. One pixel moves a neuron only a little, because it adds just one small weight to the score. Many well-placed pixels add up. This is why 20 pixels can do what 1 pixel cannot.

ADDITIONAL EXAMPLE 10 *easy practice*

(Made-up numbers.) Now each helpful pixel adds $+0.8$ to the score of h_4, which starts at -1.52. How many pixels are needed to make h_4 fire?

ANSWER After 1 pixel the score is $-1.52 + 0.8 = -0.72$ (value 0.33, not firing). After 2 pixels it is $-1.52 + 1.6 = +0.08$ (value 0.52), so h_4 fires. Answer: 2 pixels.

Topic 11 Strategy 2: inspect the weights

IN PLAIN WORDS

Strategy 1 needs thousands of tries. Strategy 2 is cheaper: **read the weights and think first**. The wiring picture of Topic 9 already tells us a lot. The output weights are $+6.96$, $+2.57$, -8.44, -8.93. To lower $P(Y=8)$ we can do one of two things:

- switch **off** a supporter (h_1 or h_2), or
- switch **on** a veto (h_3 or h_4).

The slide asks: **"Which neuron would you go after?"** (In the lab there is a button "Go after h_1" ... "Go after h_4".) To choose well, ask three questions about a neuron:

1. **Does it matter?** A big output weight means a big effect.
2. **Is it easy to move?** A neuron whose score is close to the border (score 0, value 0.5) needs only a small push.
3. **Which way must it go?** Supporters must go down, vetoes must go up.

DAILY-LIFE PICTURE

A tug-of-war: two players (h_1, h_2) pull towards "8". Two other players (h_3, h_4) pull against it, and they hold *thick, strong ropes*, but they are standing still because they are a little behind the line. To change the winner cheaply, do not argue with the strongest puller of the winning team. Instead help a player with a thick rope who is already close to the line: a small push makes him pull, and his pull is huge.



"How the network decides" (redrawn from the slide). Pink = a must-have neuron fires. Blue = a veto fires. An 8 needs h_1 and h_2 on and no blue circle filled.

QUESTION 21 *how the network decides*

Look at the four columns of the figure. For each one, say which neurons fire, whether the rule for "8" holds, and what the network says. Which column answers the question from Topic 5, "which neurons fire for a 3?"

SOLUTION

Step 1: Read each column.

picture	h_1	h_2	h_3	h_4	$P(Y=8)$	network says
Lesson 11, an 8	on	on	off	off	0.99	8
Lesson 11, a 3	on	on	**on**	off	0.03	not 8
clean 8	on	on	off	off	0.98	8
adversarial 8	on	on	off (0.32)	**on** (0.60)	0.10	not 8

Step 2: What is the pattern?

In *all four* columns h_1 and h_2 fire. The only thing that changes the answer is the vetoes: a fired veto (h_3 for the 3, h_4 for the adversarial 8) turns the answer into "not 8".

ANSWER The second column (the 3) answers the Topic 5 question: for a 3, h_1, h_2 *and the veto* h_3 fire, so $P(Y=8) = 0.03$. The adversarial 8 fails the rule in the same way, but through the other veto, h_4.

Note. This is the most important sentence of the section so far: the adversarial 8 fools the network by making a veto fire, exactly as a real 3 would.

QUESTION 22 *check the slide*

The slide says the adversarial 8 gives $P(Y=8) = 0.10$. Use the output weights ($+6.96$, $+2.57$, -8.44, -8.93) and the output neuron's bias (≈ -3.29, found in Topic 9) to check this number from the neuron values 1.00, 0.87, 0.32, 0.60.

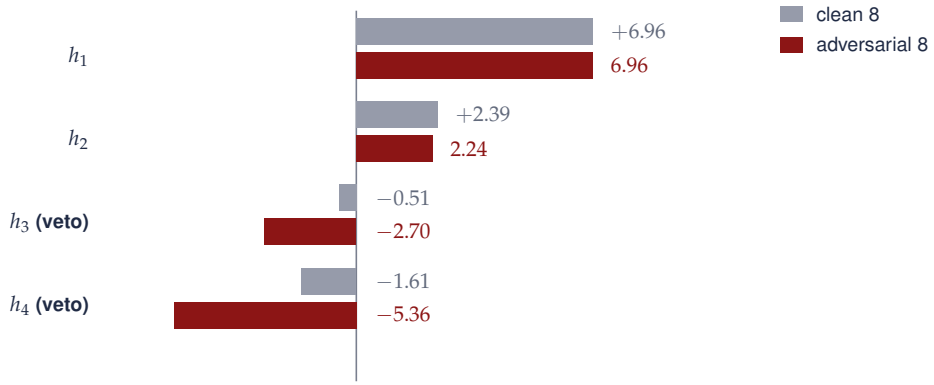
SOLUTION

Step 1: Votes of the adversarial 8.

neuron	calculation	vote	vote for the clean 8
h_1	$1.00 \times (+6.96)$	$+6.96$	$+6.96$
h_2	$0.87 \times (+2.57)$	$+2.24$	$+2.39$
h_3	$0.32 \times (-8.44)$	-2.70	-0.51
h_4	$0.60 \times (-8.93)$	-5.36	-1.61
total		$\mathbf{+1.14}$	$\mathbf{+7.24}$

Step 2: Add the bias and squash.

$$z \approx 1.14 - 3.29 = -2.15 \qquad P(Y=8) = \frac{1}{1+e^{2.15}} \approx 0.10$$



The votes of the four hidden neurons for the clean 8 (grey) and for the adversarial 8 (red). The two vetoes now vote strongly against 8.

ANSWER The total vote falls from $+7.24$ to $+1.14$. With the bias the score becomes -2.15, and $P(Y=8) \approx 0.10$, exactly as on the slide. So the model of Topics 2 and 9 really explains the network.

QUESTION 23 *which neuron would you go after?*

The slide asks: "Which neuron would you go after?" For each hidden neuron of the clean 8, work out (a) how far its score is from the border 0, and (b) what $P(Y=8)$ would be if it switched *completely* (the supporters fully off, the vetoes fully on) while everything else stays the same. Then choose.

SOLUTION

Step 1: The score of each neuron.

We read the score from the value with the S-curve run backwards, $\ln\big(h/(1-h)\big)$. The value 1.00 is rounded, so for h_1 we only know the score is above $+5$.

Step 2: What if it switched completely?

Start from the score $+3.94$ of the output neuron and change one vote only. For example, if h_1 went fully off we lose its vote $+6.96$, and the score becomes $3.94 - 6.96 = -3.02$.

neuron	value now	output weight	distance to border	$P(Y=8)$ if it switches fully
h_1 (supporter)	1.00	$+6.96$	more than 5	0.05 (fully off)
h_2 (supporter)	0.93	$+2.57$	2.6	0.83 (fully off)
h_3 (veto)	0.06	-8.44	2.8	0.02 (fully on)
h_4 (veto)	0.18	-8.93	1.5	0.03 (fully on)

(Our calculation from the slide numbers.)

Step 3: Decide.

- h_2: even if it switched completely off, $P(Y=8)$ stays at 0.83 and the network still says 8. Its weight is too small to matter.
- h_1: it would work, but it sits far from the border (score above 5). A huge push would be needed, and the S-curve is nearly flat there.
- h_3 and h_4: both have a big weight *and* both are not far from the border. h_4 is the closest (1.5).

ANSWER Go after a veto. h_4 is the best choice: a big weight (-8.93) and the smallest distance to the border (1.5). h_3 is a close second.

Note. This is the **aha moment** of the section: *the vetoes are the easiest neurons to change. They are not far from switching on, and they have big weights into the output.*

ADDITIONAL EXAMPLE 11 *easy practice*

(Made-up numbers.) A network has two vetoes. Veto A has output weight -9 and its score is -0.5. Veto B has output weight -9 and its score is -6. Which would you go after? What if Veto C has weight -1 and score -0.5?

ANSWER A and B have the same big weight, but A is much closer to the border (0.5 away instead of 6), so A is easier to switch on: choose A. Veto C is just as close as A, but its weight is only -1, so even when it fires it changes the answer very little: A is still the better target.

Topic 12 What makes h_4 fire? Reading a weight picture

IN PLAIN WORDS

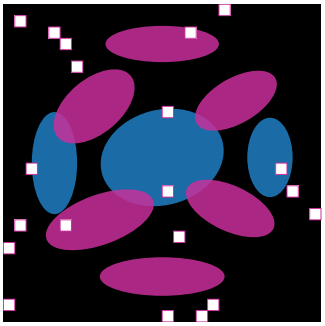
Every hidden neuron has one weight for *each of the 784 pixels*. If we put these 784 weights back into a 28×28 grid and colour them, we get a **weight picture**:

- **Pink**: a positive weight. Ink on this pixel pushes the neuron **up**.
- **Blue**: a negative weight. Ink on this pixel pushes the neuron **down**.
- **Dark**: a weight close to zero. Ink here hardly matters.

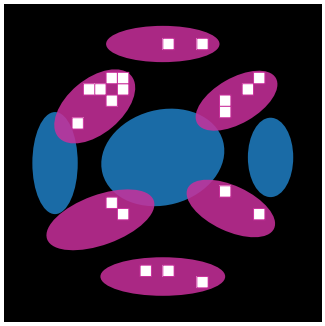
The brighter the colour, the bigger the push. So a weight picture is a **map of sensitive spots**: it shows where one white pixel does the most for this neuron.
The slide "What makes h_4 fire?" shows the weight pictures of h_4 and h_3 with the 20 added pixels of the adversarial 8 marked on them. For *both* pictures it says: **"20 of 20 on pink"**. Every one of the 20 pixels sits on a pink spot.

DAILY-LIFE PICTURE

Imagine a wall with hot and cold spots (a radiator behind it). If you touch hot spots, you pull your hand back (up). If you touch cold spots, you relax (down). If you touch twenty *random* spots, some are hot and some are cold, so they roughly cancel and you feel almost nothing. But if you touch twenty *hot* spots, you jump. The map tells you where to touch.



20 random pixels
spread over pink, blue and dark



20 chosen pixels
20 of 20 on pink

A schematic weight picture (pink = pushes the neuron up, blue = pushes it down, dark = hardly matters). Left: random pixels land everywhere. Right: the chosen pixels all land on pink. This is only a sketch; the real weight pictures of h_4 and h_3 are on the slide and in the live lab.

QUESTION 24 *reading the colours*

(a) What does a white pixel on a pink spot of h_4's weight picture do to h_4? (b) On a blue spot? (c) On a dark spot? (d) Why is "20 of 20 on pink" such a strong clue?

SOLUTION

(a), (b), (c)

Pink: h_4 goes **up**. Blue: h_4 goes **down**. Dark: almost nothing happens.

(d)

Suppose we had put the 20 pixels at random. They would land on pink, blue and dark spots in a mixed way, and the ups and downs would roughly cancel. For *all* 20 to land on pink is no accident: they were placed to push the vetoes up, and each of them does so. The weight picture *explains* why the adversarial 8 works, and it also tells us where to look for such pixels.

ANSWER Pink: up. Blue: down. Dark: no effect. "20 of 20 on pink" means every added pixel pushes the vetoes h_4 and h_3 up, so their pushes add up instead of cancelling.

QUESTION 25 *random against chosen*

(Made-up numbers, for illustration.) Say each pixel on a pink spot adds $+0.1$ to h_4's score, each pixel on a blue spot adds -0.1, and dark spots add 0. Say the picture is 30% pink, 30% blue and 40% dark. What happens to h_4 (it starts at 0.18, score -1.52) with 20 random pixels, and with 20 pixels on pink?

SOLUTION

Step 1: 20 random pixels.

On average 30% of 20 is 6 pixels on pink, 6 on blue and 8 on dark. The net push is

$$6 \times (+0.1) + 6 \times (-0.1) + 8 \times 0 = 0.$$

So h_4 stays at about 0.18.

Step 2: 20 pixels on pink.

$$20 \times (+0.1) = +2.0, \qquad \text{new score} = -1.52 + 2.0 = +0.48, \qquad h_4 = \frac{1}{1+e^{-0.48}} \approx 0.62.$$

ANSWER Random pixels: no change ($h_4 \approx 0.18$). Pixels on pink: the score rises by 2.0 and $h_4 \approx 0.62$, so it fires. The real slide shows $h_4 = 0.60$ for the adversarial 8, a very similar number.

Note. Now we can say more about the question from Topic 7, "You'd still read an 8. Why doesn't it?" You read the *shape*. The network adds up *weighted pixel votes*. Random pixels cancel out. Chosen pixels all vote the same way and add up.

ADDITIONAL EXAMPLE 12 *easy practice*

(Made-up numbers.) Suppose each pixel on pink adds $+0.2$ to h_4's score, which starts at -1.52. How many pink pixels make h_4 fire?

ANSWER We need $0.2 \times n \geq 1.52$, so $n \geq 7.6$. Check: 7 pixels give $-1.52 + 1.4 = -0.12$ (value 0.47, not firing); 8 pixels give $-1.52 + 1.6 = +0.08$ (value 0.52, fires). Answer: 8 pixels.

Topic 13 Ink the pinkest pixels

IN PLAIN WORDS

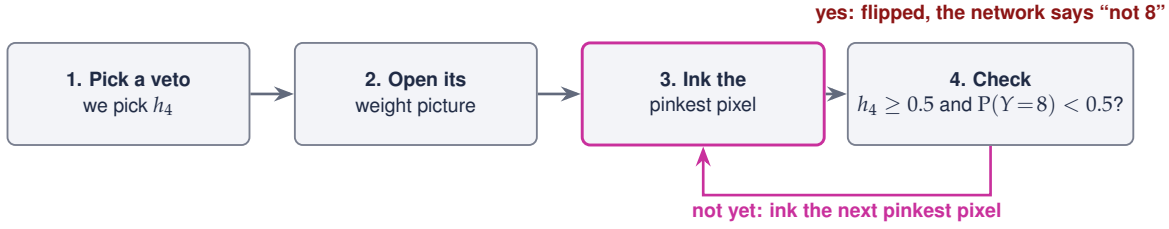
Now we turn the weight picture into a **recipe**. We do not try every pixel. We just read the map:

1. **Pick a veto.** We pick h_4.
2. **Open its weight picture.**
3. **Ink the pinkest pixel**, the one with the biggest positive weight. This pushes h_4 up the most.
4. **Check.** Does h_4 fire (0.5 or more)? Has $\mathrm{P}(Y=8)$ dropped below 0.5? If not, ink the next pinkest pixel.

The slide "Ink the pinkest pixels" shows this in the lab. It starts with an 8 where $h_4 = 0.15$ and $\mathrm{P}(Y=8) = 0.979$, and the network says 8. The button **"Ink the pinkest pixel"** adds one pixel at a time. (There is also "Reset" and a counter "pixels: 0".) The legend reads: pink: h_4 goes up if the pixel is white, blue: h_4 goes down if the pixel is white.
According to the course notes, the lab uses two 8s. Inking the pinkest pixels of h_4 flips the **clean 8 at 17 pixels** and the **fresh 8 at 18 pixels**, and h_4 fires both times. (The 8 on this slide has $h_4 = 0.15$ and $\mathrm{P}(Y=8) = 0.979$, while the wiring picture had 0.18 and 0.981, so it is a slightly different picture.)

DAILY-LIFE PICTURE

You must fill a bucket until it overflows, and you have cups of different sizes. The fastest plan is not to test every cup. You simply pick the *biggest* cup first, then the next biggest, and so on, until the bucket overflows. The weight picture is the list of cup sizes, and "overflow" means h_4 reaches 0.5.



The recipe of Strategy 2. Notice that it needs no trial runs: only one weight picture is read.

TRY IT IN THE LIVE LAB

In the lab **"Ink the pinkest pixels"** there are two 8s. Ink the clean 8 until it flips. Then press → to bring in the fresh 8 and see whether the same recipe works on it.

QUESTION 26 *how big is a pink pixel?*

In the lab, h_4 starts at 0.15. The notes say the clean 8 flips after 17 pixels, with h_4 firing. Find h_4's starting score. About how much must each pink pixel add to h_4's score, *on average*?

SOLUTION

Step 1: Starting score.

$\ln(0.15/0.85) \approx -1.74$.

Step 2: What must happen.

h_4 fires when its value is 0.5 or more, that is, when its score reaches at least 0. So the 17 pixels together must add at least 1.74.

Step 3: Average per pixel.

$$\frac{1.74}{17} \approx 0.10$$

ANSWER The starting score is about -1.74, and each pink pixel adds at least about 0.10 on average. This is exactly the size we used in the made-up example of Topic 12.

Note. This is our estimate from the slide values and the course notes. The first pixels inked are the pinkest, so they add more than the last ones.

QUESTION 27 *does it work on a fresh 8?*

The clean 8 flips at 17 pixels and the fresh 8 at 18. Why can the *same* recipe work on a second picture? Why is that a good sign for the strategy?

SOLUTION

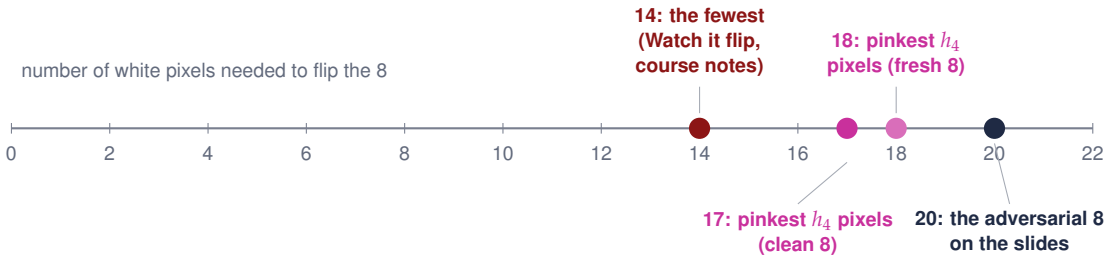
Step 1: Whose map is it?

A weight picture belongs to the *network*, not to the picture we give it. The pink spots of h_4 are the same spots whichever 8 we show.

Step 2: What this means.

So reading the map once gives us a recipe for *any* 8, not a trick for one single picture. That is why the fresh 8 needs almost the same number of pixels (18 against 17).

ANSWER Because the weights belong to the network and not to the picture, the same pink spots work for any 8. A recipe that works on a fresh picture shows that we have found a real weakness of the network, and not just a lucky trick for one image.



How many pixels each approach needs. Fewer is a stronger attack, but the cheapest way to get there is not the same as the best way.

QUESTION 28 *which strategy is better?*

Compare "try everything, worst pixel first" (14 pixels, the number in the course notes) with "ink the pinkest pixels of h_4" (17 pixels for the clean 8). Which is better? How many more pixels does the second need?

SOLUTION

Step 1: The difference.

$17 - 14 = 3$ more pixels, which is about 21% more ($3 \div 14 \approx 0.21$).

Step 2: What each method needs.

method	pixels	what it needs
Try everything (worst pixel first)	14	A trial run for every pixel at every step: about 9,000 runs (Question 18).
Ink the pinkest h_4 pixels	17 (clean), 18 (fresh)	Reading one weight picture. No trial runs.

Step 3: Decide.

If only the number of pixels counts, try everything wins. But Strategy 2 pays only three extra pixels and saves thousands of trial runs, it *explains* why the attack works, and it works on a fresh picture too.

ANSWER Try everything is a little stronger (14 against 17 pixels, 3 fewer), but Strategy 2 is far cheaper, easier to understand and works on new pictures. Neither is simply "better"; they trade strength against cost.

ADDITIONAL EXAMPLE 13 *easy practice*

A friend says: "Strategy 2 is worse, because it needs 17 pixels and Strategy 1 needs only 14." Give two reasons why Strategy 2 is still very useful.

ANSWER (1) It is cheap: it reads one weight picture instead of running the network about 9,000 times. (2) It explains *why* the digit flips (a veto is pushed over the line), and the same recipe works on a fresh 8 (18 pixels), so it finds a real weakness of the network.

Topic 14 Strategy 3: use the slope

IN PLAIN WORDS

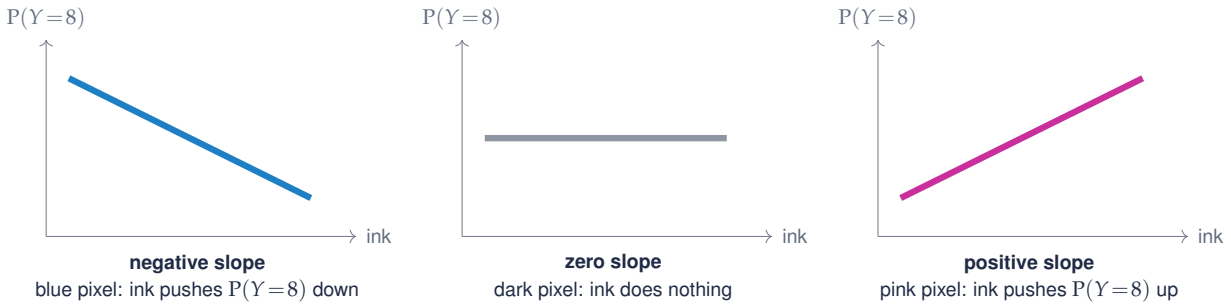
Strategy 3 asks the computer one clever question about a pixel: **"If I make this pixel a tiny bit whiter, does $P(Y=8)$ go up or down, and how fast?"** The answer is called the **slope** of $P(Y=8)$ at that pixel.

- A **negative** slope means: ink here pushes $P(Y=8)$ **down**. This is what we want.
- A **positive** slope means: ink here pushes $P(Y=8)$ **up**. We avoid these pixels.
- A slope near **zero** means: ink here does almost nothing.

The slides measure the slope of the **log likelihood** (LL, from Topic 4). The log goes up when $P(Y=8)$ goes up, so the slope of LL has the same sign as the slope of $P(Y=8)$. The sign is all we need.
You do not have to do any calculus. The computer does it on the slide. And the computer can find the slope of *every one of the 784 pixels in one go*.

DAILY-LIFE PICTURE

You are on the foggy hill of Topic 4 again, but now every pixel is a direction in which you can step. The slope tells you, for each direction, whether the ground goes down or up under your foot and how steeply. This time you want to walk *downhill* (lower $P(Y=8)$) as fast as you can, so you simply take the steepest downhill step first.
Training climbs uphill by changing the weights. The attack walks downhill by changing the pixels. It is the same tool, the slope, used in the opposite direction on a different set of knobs. The slide says it in four words: "x and θ swap places" (x is a pixel, θ is a weight).



Three kinds of pixels. Going to the right means more ink (a whiter pixel). A line that goes down is a negative slope, a line that goes up is a positive slope. The sketches show the idea, not real data.

TRY IT IN THE LIVE LAB

In the lab **"Follow the slope"** you see an 8 ($P(Y=8) = 0.979$, "says 8") next to a picture of every pixel's slope. The slide shows "slope at this pixel -0.0054" and asks: **"Which way does this pixel push $P(Y=8)$?"** In class the room only has to say whether the slope at the worst pixel is positive or negative. Then the button **"Ink the bluest pixel"** inks the pixels one by one ("Reset" starts again).

QUESTION 29 *reading a slope*

The slide "Ask the computer for the slope" says that at the worst pixel the slope is -0.0054. What does the sign tell us? Is ink at this pixel good for our attack? Why is the number so small?

SOLUTION

Step 1: The sign.

The slope is **negative**, so more ink at this pixel makes the log likelihood, and so $P(Y=8)$, **go down**.

Step 2: Good or bad for us?

We want $P(Y=8)$ to fall below 0.5, so a negative slope is exactly what we are looking for. This is a pixel worth inking.

Step 3: Why so small?

One pixel is only one tiny voice among 784, and the network is very sure at this moment ($P(Y=8) = 0.979$), which puts it on the flat part of the S-curve. We explain this in Topic 15.

ANSWER The sign is negative, so ink on this pixel pushes $P(Y=8)$ down. This is good for the attack. The number is small because a single pixel has little power and the network is very sure, so the S-curve is flat there. What counts is the sign, and the size to compare pixels with each other.

QUESTION 30 *how much came through each neuron?*

The slide splits the slope -0.0054 by neuron: "through the vetoes -0.0052", "through h_1 and h_2 -0.0002", "all four paths -0.0054". Check the sum. What share of the slope goes through the vetoes? What does this remind you of?

SOLUTION

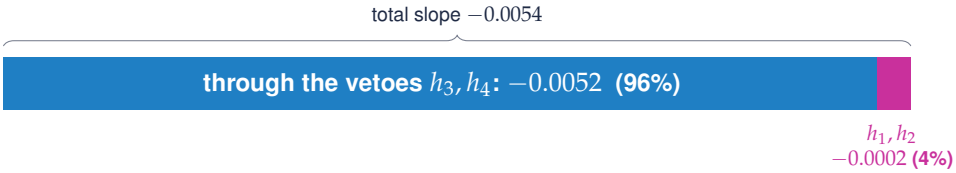
Step 1: Add the two parts.

$$-0.0052 + (-0.0002) = -0.0054$$

This is the total on the slide, so the numbers agree.

Step 2: The shares.

$$\frac{0.0052}{0.0054} \approx 0.963 \ (96\%) \qquad \frac{0.0002}{0.0054} \approx 0.037 \ (4\%)$$



ANSWER $-0.0052 - 0.0002 = -0.0054$. About 96% of the effect travels through the two vetoes. This matches Strategy 2: the vetoes are the weak spot of the network.

ADDITIONAL EXAMPLE 14 *easy practice*

(Made-up numbers.) Four pixels have these slopes: A $= +0.003$, B $= -0.0054$, C $= 0.000$, D $= -0.012$. Which pixel would you ink first, which second, and which would you never ink?

ANSWER Ink the most negative (the "bluest") first: D (-0.012), then B (-0.0054). C does nothing, so it is a waste. Never ink A: its slope is positive, so it would push $\mathrm{P}(Y=8)$ *up*, the wrong way.

Topic 15 The chain rule in plain words

IN PLAIN WORDS

How does ink on a pixel reach $\mathrm{P}(Y=8)$? Through a **chain**. On the slide the pixel x goes into the neuron h_4 (through a weight θ), h_4 goes into the output $p = \mathrm{P}(Y=8)$ (through the weight $\theta_4 = -8.93$), and p gives the log likelihood LL.

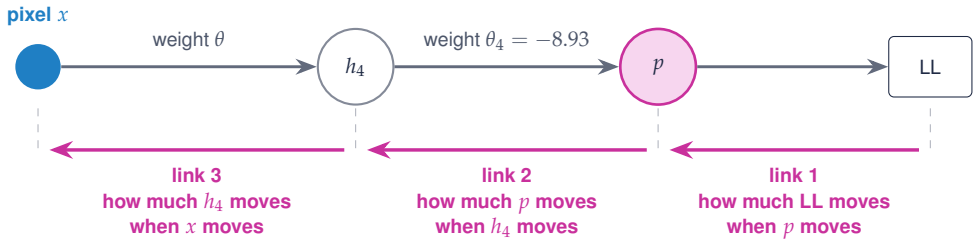
The **chain rule** says: *the effect of the whole chain is the product of the effects of its links.* Each link answers a small question: "if this thing changes a little, how much does the next thing change?" The slide writes it with the symbol ∂ (read it as "a tiny change in"):

$$\frac{\partial \mathrm{LL}}{\partial x} = \frac{\partial \mathrm{LL}}{\partial p} \cdot \frac{\partial p}{\partial h_4} \cdot \frac{\partial h_4}{\partial x}.$$

The slide shows **one path**, the one through h_4. There are four hidden neurons, so there are four paths. The full slope of a pixel is the *sum* of the four path-slopes (this is the split -0.0052 and -0.0002 of Topic 14).

DAILY-LIFE PICTURE

In a barter market a trader gives you 2 oranges for 1 apple. Another trader gives 3 bananas for 1 orange. A third gives $\frac{1}{2}$ mango for 1 banana. How many mangoes can one apple buy through the chain? You *multiply* the three rates: $2 \times 3 \times \frac{1}{2} = 3$ mangoes. The chain rule is exactly this idea, but with "how much does the next thing change" in place of exchange rates.



One path of the chain, through h_4. We go forward (grey arrows) to compute $\mathrm{P}(Y=8)$, then backward (pink arrows) to find how much each link matters. The slope of the pixel is link 1 $\times$ link 2 $\times$ link 3.

QUESTION 31 *the barter chain*

One apple buys 2 oranges, one orange buys 3 bananas, and one banana buys $\frac{1}{2}$ mango. How many mangoes does one apple buy? What happens if the last trader turns things the other way round (a *negative* rate)?

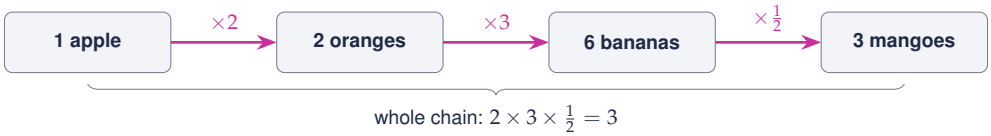
SOLUTION

Step 1: Follow the chain.

1 apple $\to$ 2 oranges $\to 2 \times 3 = 6$ bananas $\to 6 \times \frac{1}{2} = 3$ mangoes.

Step 2: A negative link.

If one link turned "more" into "less" (a negative rate), the whole chain would turn more into less. That is what the weight $\theta_4 = -8.93$ of a veto does: pushing h_4 *up* pushes $\mathrm{P}(Y=8)$ *down*.



ANSWER One apple buys 3 mangoes. The effect of a chain is the product of its links, and a negative link reverses the direction of the whole chain.

QUESTION 32 *the three links of the slide*

The slide gives the three links of the path through h_4: link $1 = 1/p$, link $2 = p(1-p) \times \theta_4$, link $3 = h_4(1-h_4) \times \theta$. (Made-up number: the pixel's weight into h_4 is $\theta = +0.2$.) Take $p = 0.979$, $h_4 = 0.15$ and $\theta_4 = -8.93$ from the slides. Work out the slope of the path.

SOLUTION

Step 1: Link 1.

$$\frac{1}{p} = \frac{1}{0.979} \approx 1.02$$

Step 2: Link 2.

$$p(1-p)\,\theta_4 = 0.979 \times 0.021 \times (-8.93) \approx -0.184$$

Step 3: Link 3.

$$h_4(1-h_4)\,\theta = 0.15 \times 0.85 \times 0.2 = 0.0255$$

Step 4: Multiply.

$$1.02 \times (-0.184) \times 0.0255 \approx -0.0048$$

(Because p cancels, there is a shorter form: slope $= (1-p)\,\theta_4\,h_4(1-h_4)\,\theta$. Check: $0.021 \times (-8.93) \times 0.1275 \times 0.2 \approx -0.0048$.)

ANSWER The slope through h_4 is about -0.0048. It is negative, because exactly one link (θ_4) is negative. It is about the same size as the -0.0052 that the slide shows for both vetoes together, so a pixel weight of about 0.2 is reasonable. (Our estimate; the weight 0.2 is made up.)

QUESTION 33 *why is the slope small when the network is sure?*

The factor $p(1-p)$ measures how steep the S-curve is. Work it out for $p = 0.5$ and $p = 0.979$ and compare. What does this say about the first pixels we ink and the later ones?

SOLUTION

Step 1: The two values.

$$0.5 \times 0.5 = 0.25 \qquad\qquad 0.979 \times 0.021 \approx 0.0206$$

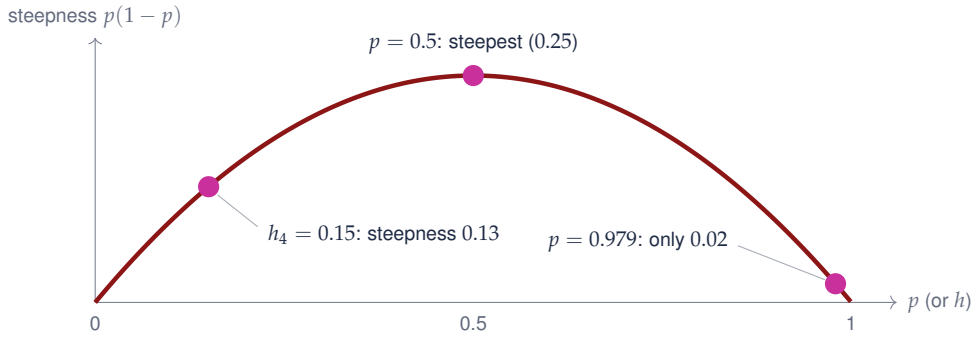
Step 2: Compare.

$$\frac{0.25}{0.0206} \approx 12$$

Step 3: What it means.

When the network is very sure ($\mathrm{P}(Y=8) = 0.979$) the S-curve is 12 times flatter than in the middle, so every pixel looks weak. *Our reading:* as ink pushes $\mathrm{P}(Y=8)$ down towards 0.5 the curve gets steeper, so the later pixels have a bigger effect than the first ones.

ANSWER 0.25 against 0.0206: the curve is about 12 times flatter at 0.979. That is why the slope of -0.0054 is small, and why the first pixels seem to do so little.



How steep the S-curve is at different values. It is steepest in the middle and almost flat near 0 and near 1. At $p = 0.979$ the curve is about 12 times flatter than at $p = 0.5$.

ADDITIONAL EXAMPLE 15 *easy practice*

A path has three links with signs $(+)$, $(-)$, $(+)$. What is the sign of the whole slope? What if the signs are $(-)$, $(-)$, $(+)$?

ANSWER Multiply the signs: $(+)(-)(+) = (-)$, so the slope is negative. For $(-)(-)(+)$ we get $(+)$, because two negatives cancel. In our path only θ_4 is negative, so the slope is negative.

Topic 16 Every pixel's slope: a picture of the whole map

IN PLAIN WORDS

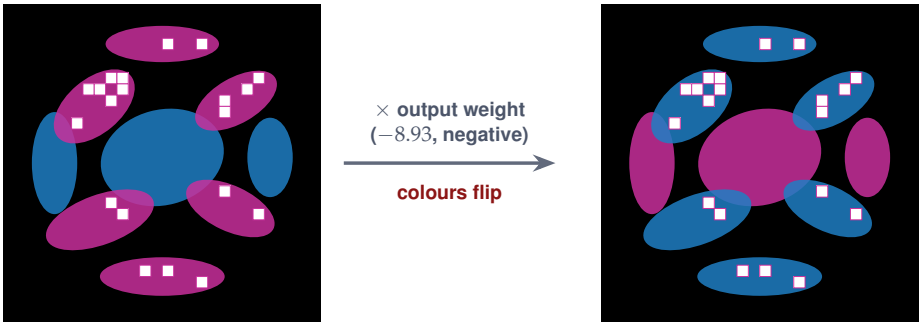
Now ask the computer about *all 784 pixels at once* and draw the answers as a picture. This is the slide **"Every pixel's slope"**:

- **Pink**: $\mathrm{P}(Y=8)$ goes **up** if this pixel is white.
- **Blue**: $\mathrm{P}(Y=8)$ goes **down** if this pixel is white.
- **Dark**: almost no effect.

On the slide the boxes mark the pixels you have inked. The bluest pixels are the best places for ink, and the button **"Ink the bluest pixel"** (Topic 14) adds them one at a time.
The course notes point out something beautiful: *the blue of the slope picture lines up with the pink of the vetoes' weight pictures.* The next question explains why.

DAILY-LIFE PICTURE

A traffic app colours every road of a city: red roads are slow, green roads are fast. You do not need to drive on every road to find the slow ones. One look at the map, which the app computed for all roads at once, tells you everything. The slope picture is the same kind of map for the pixels.



a veto's weight picture
pink: ink pushes the veto up

the slope picture
blue: ink pushes $\mathrm{P}(Y=8)$ down

A schematic sketch (not the real pictures). The same twenty pixels sit on pink in the veto's weight picture and on blue in the slope picture, because the veto's output weight is negative.

QUESTION 34 *why do the colours flip?*

A pixel is pink in the weight picture of the veto h_4. Why is the same pixel blue in the picture of every pixel's slope?

SOLUTION

Step 1: Follow the push.

Ink on a pink pixel of h_4 pushes h_4 **up** (Topic 12).

Step 2: The veto's output weight.

h_4 reaches the output with the weight -8.93, which is negative. A higher h_4 therefore pushes $\mathrm{P}(Y=8)$ **down**.

Step 3: Put them together.

Up then down gives down: ink on this pixel lowers $\mathrm{P}(Y=8)$, so its slope is negative, which is drawn in blue. For a supporter (h_1 or h_2) the output weight is positive and the colours would *not* flip.

ANSWER Because the output weight of the veto is negative. "Pushes the veto up" becomes "pushes $\mathrm{P}(Y=8)$ down", so pink in the veto's weight picture becomes blue in the slope picture.

QUESTION 35 *why do the vetoes carry the slope?*

On the slide "Every pixel's slope" the hidden neurons are $h_1 = 0.99$, $h_2 = 0.95$, $h_3 = 0.11$, $h_4 = 0.15$. Each neuron's path contributes (output weight) $\times$ (steepness $h(1-h)$ of its S-curve) $\times$ (the pixel's weight into that neuron). Compare the first two factors for the four neurons.

SOLUTION

Step 1: The table.

neuron	value h	steepness $h(1-h)$	output weight	weight $\times$ steepness
h_1	0.99	0.0099	$+6.96$	$+0.069$
h_2	0.95	0.0475	$+2.57$	$+0.122$
h_3 (veto)	0.11	0.098	-8.44	-0.826
h_4 (veto)	0.15	0.1275	-8.93	-1.139

(Our calculation from the slide numbers.)

Step 2: Read the table.

The supporters sit near 1, where the S-curve is almost flat, so their steepness is tiny. The vetoes sit near 0.1, where the curve is much steeper, and they also have the biggest output weights. Their factors are about 10 times bigger: $(0.826 + 1.139) \div (0.069 + 0.122) \approx 10$.

Step 3: Compare with the slide.

If a pixel had the same weight into all four neurons, the vetoes would carry 91% of the slope. The slide shows 96% for its worst pixel.

ANSWER The vetoes have bigger output weights *and* sit on a steeper part of the S-curve, so their factors are about 10 times larger than those of h_1 and h_2. That is why about 96% of the slope goes through them. The slope method finds the same weak spot as Strategy 2.

ADDITIONAL EXAMPLE 16 *easy practice*

(Made-up.) In a slope picture, pixel P is bright pink, pixel Q is deep blue, pixel R is dark and pixel S is light blue. Which pixel would you ink first? Which would hurt the attack?

ANSWER Ink Q first (deep blue means a strongly negative slope), then S. R does nothing. P would hurt the attack: bright pink means ink there pushes $P(Y=8)$ *up*.

Topic 17 Try everything against calculus: all three strategies work

IN PLAIN WORDS

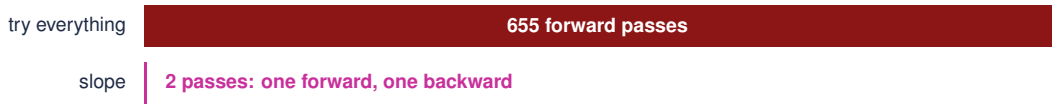
The slide **"Try Everything vs Calculus"** puts the two methods side by side: **try everything: 655 tries**, and **every pixel's slope: one pass each way**. Both pick **the same most confusing pixel**. Two new words:

- A **forward pass** runs the network from the pixels to $P(Y=8)$. One "try" of Strategy 1 is one forward pass.
- A **backward pass** goes the other way, from $P(Y=8)$ back to the pixels, and gives the slope of *all* pixels at once. It is the same backward pass that trains the weights (Topic 4). This is the second aha moment: *the backward pass that trains the weights also gives a slope for every pixel.*

So the slope method needs one forward pass and one backward pass ("one pass each way") where try everything needs 655 forward passes.

DAILY-LIFE PICTURE

You want to know which item made your restaurant bill so big. Method 1: ask the waiter to print the bill again 655 times, each time without one item. Method 2: read the itemised bill once. Both tell you which item is the most expensive, but one of them takes a minute and the other takes all day.



The cost of finding the most confusing pixel. The pink bar is so short that it is almost invisible: that is the point.

strategy	idea	cost	needs
1. Try everything	Test every pixel with the network, keep the worst.	655 tries for one search.	Only the network.
2. Inspect the weights	Pick the veto h_4, ink the pinkest pixels.	Reading one picture.	The weight picture.
3. Use the slope	Ask for every pixel's slope, ink the bluest.	One pass each way.	A backward pass.

QUESTION 36 *655 against 2*

Try everything needs 655 forward passes for one search. The slope method needs one forward and one backward pass. About how many times fewer passes is that? (Assume a backward pass costs about the same as a forward pass.)

SOLUTION

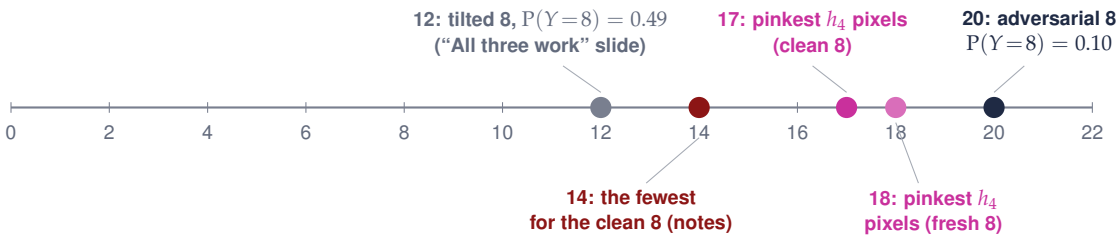
Step 1: Count the passes.

Try everything: 655. Slope: $1 + 1 = 2$.

Step 2: Divide.

$$\frac{655}{2} = 327.5 \approx 330$$

ANSWER The slope method needs about 330 times fewer passes. (Our calculation; it assumes that the backward pass costs about as much as the forward pass.) Both find the same most confusing pixel.



All the pixel counts of this handout on one line. They belong to different pictures and methods, so they cannot be ranked simply.

QUESTION 37 *12 or 14?*

On the slide "All three strategies work!" one 8 flips with 12 pixels ($0.98 \to 0.49$) and another with 20 pixels ($0.98 \to 0.10$). The course notes say 14 is the fewest for the clean 8. A student asks: "So is it 12 or 14?" How do you answer?

SOLUTION

Step 1: Are they the same picture?

No. The 12-pixel example on the slide is a tilted 8, and the 20-pixel example is the upright clean 8 (the one with $P(Y=8) = 0.10$ in Topic 7). *Our reading from the slides:* different pictures have different minimums.

Step 2: Look at where they end.

The 12-pixel picture ends at 0.49, only just below the flip line 0.5. The 20-pixel picture ends at 0.10, far below it.

Step 3: The notes.

The notes say 14 is the fewest *for the clean 8 with white pixels only*. The 20 on the slide is chosen so that we can compare with 20 random pixels (Topic 7).

ANSWER Both are right: they are different pictures. The minimum depends on the picture and on the rules (white pixels only). The 12 ends just under the line (0.49), the 20 ends far under it (0.10).

ADDITIONAL EXAMPLE 17 *easy practice*

(Made-up numbers.) A picture has 5,000 pixels. About how many forward passes does try everything need for one search? How many passes does the slope method need? How many times fewer is that?

ANSWER Try everything: about 5,000 passes. Slope: 2. That is $5{,}000 \div 2 = 2{,}500$ times fewer. The bigger the picture, the more the slope method wins.

Topic 18 The close: what does it think an 8 is? Where does this matter?

18.1 What does it think an 8 is?

IN PLAIN WORDS

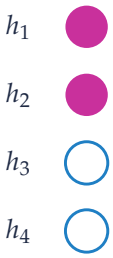
Now we turn the game around. Instead of making an 8 *fail*, we make the network *as happy as possible*. We **start with a blank (black) picture**. Then we change every pixel in the direction that raises $P(Y=8)$, as far as we like (**no limit**). The slide asks: **"What will it draw?"**
The result: $P(Y=8) = 1.00$. The neurons h_1 and h_2 are on, the vetoes h_3 and h_4 are off, so the network is completely sure. But the picture is *not an 8*: it is a messy mix of white patches. The slide ends with the question **"Who told it this isn't an 8?"**
Here is a way to see it. The network's idea of an 8 is just the short checklist of Topic 5: h_1 on, h_2 on, h_3 off, h_4 off. The messy picture ticks every box. And nobody ever showed the network such a picture during training, or told it that it is not an 8. (Our reading of the slide.)

DAILY-LIFE PICTURE

A bouncer at a gate checks only four things: a cap, a ticket, no sandals, no bag. A person wearing a cap, holding a ticket, wearing shoes and carrying nothing gets in. So does a person with a cap *drawn on paper* and a *fake* ticket. The bouncer is perfectly sure, because every box is ticked. Nobody ever taught him what a *real* cap is.



what it “draws” (a sketch)
$P(Y=8) = 1.00$



must-haves: both on

vetoes: both off

all boxes ticked: it says “8”

Left: a sketch of the kind of picture the climb produces (the real one is on the slide). Right: the neurons for that picture. Filled = fires.

QUESTION 38 *reading the slide*

For the picture it “draws”, the slide shows h_1 and h_2 filled and h_3 and h_4 empty, with $P(Y=8) = 1.00$. Does it follow the rule for “8”? How is this different from the adversarial 8?

SOLUTION

Step 1: Check the rule.

The rule needs h_1 on, h_2 on, h_3 off and h_4 off. All four are true, so the network says 8 with full confidence.

Step 2: Compare.

The adversarial 8 *looks* like an 8 to us but breaks the rule (the veto h_4 fires, $P(Y=8) = 0.10$). The blob *does not look* like an 8 but satisfies the rule ($P(Y=8) = 1.00$).

ANSWER Yes, it follows the rule. It is the mirror image of the adversarial 8: that one looks right to us but fails the network’s checklist, and the blob looks wrong to us but passes the network’s checklist.

QUESTION 39 *who told it this isn’t an 8?*

Answer the slide’s last question in your own words: why is the network sure about a picture that is clearly not an 8?

SOLUTION

Step 1: What did the network learn from?

Only from training digits with their correct labels (Topic 4). It learned to give a high $P(Y=8)$ to *digits* that are 8s and a low one to other digits.

Step 2: What did the climb do?

It simply followed the slope upwards without any limit. The slope knows only "this pixel raises $P(Y=8)$", not "this looks like an 8".

Step 3: So who told it?

Nobody. Such pictures were never in the training data, so the network's answers there were never checked or corrected.

ANSWER Nobody ever did. The network learned only from digit-like pictures. Far from them its answer was never trained or checked, and the climb pushes the checklist as far as it will go. A network can be extremely sure and still wrong.

ADDITIONAL EXAMPLE 18 *easy practice*

The bouncer's checklist is: cap, ticket, shoes, no bag. Person A has all four for real. Person B has a drawn cap and a fake ticket, but shoes and no bag. Who gets in? What does this show about the network?

ANSWER Both get in, because both tick every box. The checklist cannot tell real from fake. In the same way, the network cannot tell a real 8 from a blob that ticks its four boxes.

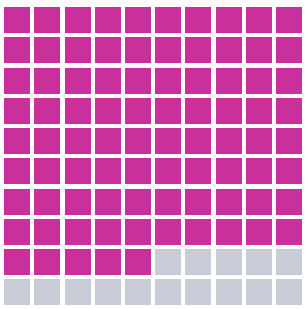
18.2 Where would a fooled network matter?

IN PLAIN WORDS

The slide "Where would a fooled network matter?" gives three real stories:

1. **Stop sign → "Speed Limit 45" (2018).** Stickers on a real sign. Seen from a moving car, **85% of the video frames were fooled**.
2. **Turtle → rifle (2018).** A 3D-printed turtle that an image network consistently called a rifle.
3. **100 poisoned pictures → broken prompt (2024).** The attack hits the *training data*: fewer than 100 corrupted pictures were enough to corrupt a Stable Diffusion prompt.

Then comes an open question for you: **"Where else would this matter to you?"**



Stop sign with stickers (2018)

100 video frames from a moving car:

85 fooled

15 read correctly

Turtle → rifle (2018): the input is changed.

100 poisoned pictures (2024): the training data is changed.

Each pink square is one video frame that was fooled (85 of 100). The other two stories are shown as text.

QUESTION 40 *input or training data?*

Which of the three stories change the *input* that the network sees (like our extra pixels), and which one changes the *training data*? Why is the second kind a different kind of danger?

SOLUTION

Step 1: Sort the stories.

The stop sign (stickers) and the turtle change the **input**, the picture the network is shown, just as our 20 pixels do. The 100 poisoned pictures change the **training data**.

Step 2: Why is poisoning different?

Input attacks happen *after* training, when the network is already built. Poisoning happens *before*: the network learns the wrong thing from the start (Topic 4: the weights are chosen to fit the training data). *Our reading:* so even a perfect network can be damaged without anyone touching a single weight.

ANSWER Stop sign and turtle: input attacks. The 100 poisoned pictures: a training-data attack. Poisoning corrupts what the network learns, before it is ever used.

QUESTION 41 *where else would this matter to you?*

The slide ends with: "Where else would this matter to you?" Name two places and say what could go wrong.

SOLUTION

There is no single right answer. Here are some starting ideas (ours, not from the slides):

- **Face unlock on a phone:** a printed photo or a special pattern could open it for the wrong person.
- **Medical scans:** a tiny change might hide a sign of illness from a program that helps doctors.
- **Spam and fraud filters:** a few well-chosen changes to a message might let it pass.
- **Self-driving cars:** like the stop sign, a misread sign could cause an accident.

ANSWER Any place where a network makes a decision on its own: face unlock, medical scans, spam or fraud filters, driving. In each, a small change that people do not notice could change the decision.

ADDITIONAL EXAMPLE 19 *easy practice*

(Made-up count.) A car camera sees 200 video frames of the stop sign. If 85% are fooled, how many frames are fooled and how many are read correctly? Is the car safe in the other 15%?

ANSWER $0.85 \times 200 = 170$ frames are fooled and 30 are read correctly. The 15% does not make the car safe: it must be right *every time*, and here it is wrong most of the time.

18.3 Extra time: fool the full network

IN PLAIN WORDS

"That's all folks!" ends the section. The slide **"[Extra Time] Fool the full network"** is a bonus for spare time. It uses the **full network** of Topic 3 ($784 \to 500 \to 300 \to 20 \to 10$ digits, 549,030 weights and biases). At the start the first guess is 8 (1.00) and the second guess is 3 (0.00). This time *you* choose the digit you want the network to say: the buttons 0 to 9 (the slide shows 3 selected). Then press **"One pixel"** (add one pixel) or **"Until it flips"**. A counter shows "pixels inked". Three questions are written on the slide: **"Is it still an 8?"**, **"Which digit should it say?"** and **"How many pixels will it take?"**

PLACE YOUR BET

Before you try: do you think the full network will need *more* or *fewer* pixels than the small one (clean 8: 14)? Write your bet and a reason: _____

QUESTION 42 *predict first*

Give one reason why the full network might need *more* pixels than the small network, and one reason why it might need *fewer*.

SOLUTION

More pixels.

The full network is about 175 times bigger (Topic 5). Its answer is spread over many neurons, so no small group of neurons, like our two vetoes, may decide everything. It may need more ink.

Fewer pixels.

It also has many more weights and many more paths from every pixel to the answer, so a single pixel can push on many neurons at once. A bigger network may be *easier* to nudge.

Which is true?

The slides and notes give no number for the full network, so we do not know. That is why the slide is an experiment: try it, count the pixels, and compare with the 14 of the small network.

ANSWER Both arguments are reasonable. The handout cannot say which one wins, because the slides give no count for the full network. Test it in the lab and compare your count with your bet.

That's all folks! Our three goals, answered

1. WHY DID IT MISS?

The 20 chosen pixels pushed a **veto** over the line: h_4 went $0.18 \to 0.60$ and fired, h_3 rose $0.06 \to 0.32$. The rule for "8" failed and $P(Y=8)$ fell to 0.10.

2. HOW DO WE FIND THE PIXELS?

Try everything (655 tries for one search), **inspect the weights** (ink the pinkest pixels of h_4: 17 for the clean 8), or **use the slope** (one pass each way). All three find the same weak spot: the vetoes.

3. WHAT DOES IT THINK AN 8 IS?

Just a **checklist**: h_1 and h_2 on, h_3 and h_4 off. A messy picture can tick every box and get $P(Y=8) = 1.00$, because nobody told the network otherwise.

Quick self-check

question	answer
How many pixels does a 28×28 picture have?	784.
When does a neuron fire?	When its value is 0.5 or more.
Which neurons are the vetoes, and what is the rule for "8"?	h_3 and h_4. h_1 and h_2 on, both vetoes off.
What does ink on a pink pixel of h_4's weight picture do?	It pushes h_4 up.
Why do random pixels barely change $P(Y=8)$ but chosen ones do?	Random pixels cancel out; chosen pixels all push the same way (on pink) and add up.
What does a negative slope mean?	Ink on that pixel pushes $P(Y=8)$ down.
Why does the slope method need so few passes?	The backward pass gives the slope of all pixels at once: one pass each way.
Why can the network be sure about a messy blob?	It checks only a short checklist and was never trained on such pictures.
Which real story attacks the training data?	The 100 poisoned pictures (2024).

WHAT TO REMEMBER

TOPICS 1 TO 9: THE PICTURE, THE NETWORK AND THE MYSTERY

- A picture is a list of brightness numbers. Our pictures have $28 \times 28 = 784$ pixels.
- A neuron multiplies its inputs by weights, adds a bias, and squashes the result with an S-curve into 0 to 1. It *fires* at 0.5 or more.
- A network is neurons in layers. The full network has 549,030 weights and biases, the small one has 3,145.
- The weights are learned by maximum likelihood: climb the log likelihood using the slope.
- The small network says 8 when h_1 and h_2 are on and both vetoes (h_3, h_4) are off. The network says 8 when $P(Y=8) \geq 0.5$.

- Random pixels barely matter (0.98 $\rightarrow$ 0.97). Chosen pixels do (0.98 $\rightarrow$ 0.10). The *place* matters, not the number.
- A veto has a big weight, so one veto that fires can overrule the supporters.

TOPICS 10 TO 13: STRATEGY 1 AND STRATEGY 2

- **Strategy 1, try everything:** test every pixel, keep the worst one, repeat. It flips the clean 8 with 14 pixels, but needs about 9,000 trial runs. One pixel alone barely moves $P(Y=8)$.
- Inside the network, the **vetoes** change: h_4 goes 0.18 $\rightarrow$ 0.60 (it fires) and h_3 goes 0.06 $\rightarrow$ 0.32. The adversarial 8 fools the network by making a veto fire, just as a real 3 does (h_3 fires).
- **Strategy 2, inspect the weights:** a good target has a *big output weight* and is *close to the border*. That is a veto, and h_4 is the best one.
- A **weight picture** shows one neuron's weight for every pixel: pink pushes up, blue pushes down, dark does nothing. All 20 added pixels sit on pink for h_4 and for h_3.
- Random pixels cancel out. Pixels on pink add up. This answers "You'd still read an 8. Why doesn't it?"
- **The recipe:** pick h_4, open its weight picture, ink the pinkest pixel, check, repeat. It flips the clean 8 at 17 pixels and the fresh 8 at 18. The weights belong to the network, so the recipe works on any 8.

TOPICS 14 TO 18: STRATEGY 3 AND THE WRAP-UP

- The **slope** of a pixel says which way ink pushes $P(Y=8)$: negative means down (good for the attack). The computer finds it; we only read the sign.
- The **chain rule**: the effect of a chain is the *product* of its links. A negative link (a veto's weight) flips the whole chain.
- The S-curve is flat when the network is very sure, so single pixels look weak ($p(1-p)$ is 12 times smaller at 0.979 than at 0.5).
- The **slope picture**: blue lines up with the pink of the vetoes' weight pictures. About 96% of the slope goes through the vetoes.
- **One pass each way** (one forward, one backward) against 655 tries. The backward pass that trains the weights also gives a slope for every pixel.
- Climbing $P(Y=8)$ without limit makes a messy picture with $P(Y=8) = 1.00$: the network's idea of an 8 is a checklist, and nobody told it otherwise.
- Fooling networks matters in real life: stop signs, a toy turtle, poisoned training data.

Thank you for reading, and happy section!

Sohaib Hasan · Section Leader, Stanford Probability for AI